

What is SQL?

- **SQL (Structure Query Language)** is a database language used for retrieve, manage and manipulate the data stored in relational databases (A relational database stores data in tables. Each table consists of rows (records or tuples) and columns (fields or attributes))
- SQL is the primary way to interact with RDBMS like Oracle, MySQL, PostgreSQL, SQL Server, and SQLite etc. Further these RDBMS manages and manipulates the database with the help of SQL.
- SQL was developed at IBM by Donald D. Chamberlin and Raymond F. Boyce in the early 1970s.
- This SQL version is initially called SEQUEL (Structured English Query Language).

Characteristics or Properties or Features or Advantages of SQL:

1. Simple and easy to Learn:

- SQL uses High-level language (English language) to write the commands hence it is simple and easy to learn.

2. Case insensitive:

- SQL syntax is case insensitive means uppercase and lowercase letters are treated as the same.
- Eg. **CREATE** is same as **create**

3. Quick and Efficient:

- SQL retrieves large amount of data (i.e. records) from a database very quickly and efficiently.

4. Declarative Language:

- SQL specifies **what** to do (e.g., retrieve data), not **how** to do it.
- SQL Focuses on **data and results**, not procedures hence it is declarative.

5. Portable Across RDBMS platforms:

- SQL is Standardized i.e. it is ANSI SQL, so it works on multiple RDBMS platforms like MySQL, PostgreSQL, Oracle etc. with minor syntax differences.

6. Set-Based Processing:

- Operates on **sets or bunch of data** rather than row-by-row (though some row-level control exists).

7. Data Manipulation and Definition:

- SQL uses commands for data definition, retrieval and manipulation as follow-
- **Data Definition Language (DDL)** – CREATE, ALTER, DROP
- **Data Manipulation Language (DML)** – SELECT, INSERT, UPDATE, DELETE
- **Data Control Language (DCL)** – GRANT, REVOKE
- **Data Query Language (DQL)**- SELECT
- **Transaction Control Language (TCL)** – COMMIT, ROLLBACK, SAVEPOINT, SET TRANSACTION

8. Interactive and Embedded

- SQL Can be used **Interactively** via command-line or GUI tools.
- SQL can be **Embedded** in other languages such as Python, Java etc. through libraries like JDBC, ODBC

9. Robust Query Capabilities:

- SQL Supports complex operations on database to make it as robust these operations are- **Joins, Subqueries, Grouping & aggregation, Sorting, Filtering**

10. Integrity and Security:

- SQL enforces data **integrity** with **Constraints** (e.g., PRIMARY KEY, FOREIGN KEY)
- SQL provides **Security by Permissions** (access control)

11. Transaction Management:

- SQL Supports **ACID (Atomicity, Consistency, Isolation, Durability) properties** that ensures safe and consistent data updates.

Components of SQL (Basic Structure of SQL) (Commands in SQL):

- To interact with databases there is a need to learn different commands available in SQL.
- SQL is made up of different types of commands which are grouped into following categories (Depending upon task of command):

- 1) **Data Definition Language (DDL)**
- 2) **Data Manipulation Language (DML)**
- 3) **Data Control Language (DCL)**
- 4) **Data Query Language (DQL)**
- 5) **Transaction Control Language (TCL)**

Let's see these components in details-

1) Data Definition Language (DDL):

- DDL (Data Definition Language) in SQL is used to define, create, modify, delete and manage database structures such as tables, schemas, indexes, and constraints etc.
- DDL focuses on how data is stored, not the data itself.
- These commands are used by DBA user (system user) such as database designer or application developer. Note that these commands are not used by general user.
- These commands are immediate i.e. they are not having tendency to ROLLBACK. That is, once these commands are fired then it will not rollback to the previous state. There is no effect of ROLLBACK on these commands.

❖ Key Characteristics of DDL Commands:

1) **Structure-Oriented:** DDL commands deal with the structure of the database, not the data itself.

Examples include creating, altering, or deleting tables and other objects.

2) **Auto-Commit:** DDL statements automatically commit any changes made to the database.

You cannot roll back a DDL command once executed.

3) **Implicit Transactions:** Each DDL statement is treated as a complete transaction by itself.

There's no need (or option) to manually commit or rollback.

4) **Affects Schema Objects:** DDL commands modify schema objects like tables, indexes, views, stored procedures, etc.

5) **Do Not Manipulate Data:** DDL commands do not insert, update, or delete records or data; they only define or change the structure of the database.

6) **Access Control and Privileges:** Executing DDL commands often requires higher access control and higher privileges, such as DBA users.

7) **Modifies Metadata:** DDL commands directly affect the metadata stored in the data dictionary of the DBMS.

Common DDL Commands in SQL are:

DDL Commands	Purpose
CREATE	Create <u>database objects</u> (tables, views, index etc.)
ALTER	Modify existing database structures.
DROP	Delete database objects (tables, views, index etc.)
TRUNCATE	Delete all rows from a table (faster than DELETE)
RENAME	Rename a database object. (tables, views, index etc.)

2) Data Manipulation Language (DML):

- DML commands are used to manipulate (storing, retrieving, modifying, & deleting) the data stored in database tables.
- Unlike DDL, which deals with database structure, DML focuses on actual data stored in database tables.
- DML commands are **not** auto-committed i.e. When DML commands are fired, changes are not permanent to the database; they can be rolled back.

❖ Key Characteristics of DML Commands:

1. **Data-Centric:**

- DML operates on the **data within tables**, not the schema or structure of the database.
- It is used to **insert, update, delete, or retrieve** records of the database table.

2. **Transactional Control:**

- DML statements are part of a transaction.
- Changes are **not permanent** until a COMMIT is issued.
- You can **ROLLBACK** changes if needed before committing.

3. **Requires Existing Structure:**

- DML commands operate on **existing tables** created using DDL commands.

4. Can Be Rolled Back:

- DML operations can be **undone** using ROLLBACK, as they do not auto-commit by default.

5. Affects Table Rows:

- DML commands typically **target rows**—they add, change, or remove data from them.

6. Permission-Based:

- Users need **appropriate privileges** (e.g., INSERT, UPDATE, DELETE, SELECT) to perform DML operations.

Common DML Commands in SQL are:

DML Command	Purpose
SELECT	Retrieves data from one or more tables
INSERT	Adds a new row (Record) into a table.
UPDATE	Modifies existing table data in one or more rows
DELETE	Removes rows from a table.(slower than TRUNCATE)

3) Data Control Language (DCL):

- DCL commands are used to control access of data in a database.
- They manage permissions and security levels, ensuring that only authorized users can perform specific actions.

❖ Key Characteristics of DCL Commands:

1. Permission Management:

- DCL is used to **grant or revoke user rights** and permissions on database objects like tables, views, and procedures.

2. Security-Focused:

- DCL helps maintain **data security and integrity** by controlling access to data.

3. Dependent on Privileges:

- To use DCL commands, a user typically needs **administrative privileges** (like DBA or GRANT OPTION).

4. Affects Access, Not Data or Structure:

- DCL doesn't alter data (like DML) or structure (like DDL). It only changes **who can access and what** in the database.

Common DCL Commands in SQL are:

DCL Command	Purpose
GRANT	Gives specific privileges to user
REVOKE	Removes privileges from a user

4) Data Query Language (DQL):

- DQL stands for **Data Query Language**, and it is primarily used to **retrieve data** from database objects.
- SELECT command also belongs to DQL, when SELECT is fired against a table or tables, then the result is compiled into a temporary table which is displayed at the front-end.
- DQL is technically a subset of DML, it's often treated as a separate category because of its specific purpose: **querying data without modifying it**.

Key Characteristics of DQL Commands:

1. Data Retrieval Only:

- DQL is used **only to query data**, not to insert, update, or delete it.
- It does **not change** the data or database structure.

2. Centered Around the SELECT Statement:

- The core and only DQL command is **SELECT**.
- It is used to only **fetch data** from one or more tables, views, or other database objects.

3. Non-Transactional:

- Since DQL does not modify data, it is **not part of a transaction** and does **not require COMMIT or ROLLBACK**.

4. Highly Flexible and Powerful:

- SELECT can retrieve specific columns, filter rows, join multiple tables, group results, and apply functions.

5. Does Not Require High Privileges:

- Most users with basic read access can use DQL, provided they have SELECT privileges on the target objects.

Common DCL Commands in SQL are:

DQL Command	Purpose
SELECT	Retrieves data from the database objects.

5) Transaction Control Language (TCL):

- TCL statements are **used to manage the changes made by DML statements**.
- TCL commands are used to **manage transactions** in a database. A **transaction** is a sequence of operations performed as a single logical unit of work, and **TCL ensures that these operations are executed reliably and consistently**.
- These commands can cancel changes made by DML commands using ROLLBACK and also it can make changes permanent using COMMIT.
- TCL allows several statements to be grouped together as logical transactions and allows to set a **marker (savepoint) within that transaction** so we can roll back only these statements.

❖ Key Characteristics of TCL Commands:

1. Transaction Management:

- TCL commands control **when and how changes made by DML statements are saved or undone** in the database.

2. Ensures Data Integrity:

- TCL helps to maintain **ACID properties** (Atomicity, Consistency, Isolation, Durability) of transactions.

3. Works with DML:

- TCL applies **only to DML operations** (INSERT, UPDATE, DELETE) and has **no effect on DDL or DCL** (which usually auto-commit).

4. Explicit Control of Changes:

- Changes made using DML are **not permanent (not auto commit)** until COMMIT is issued.
- You can **undo** changes using ROLLBACK before committing.

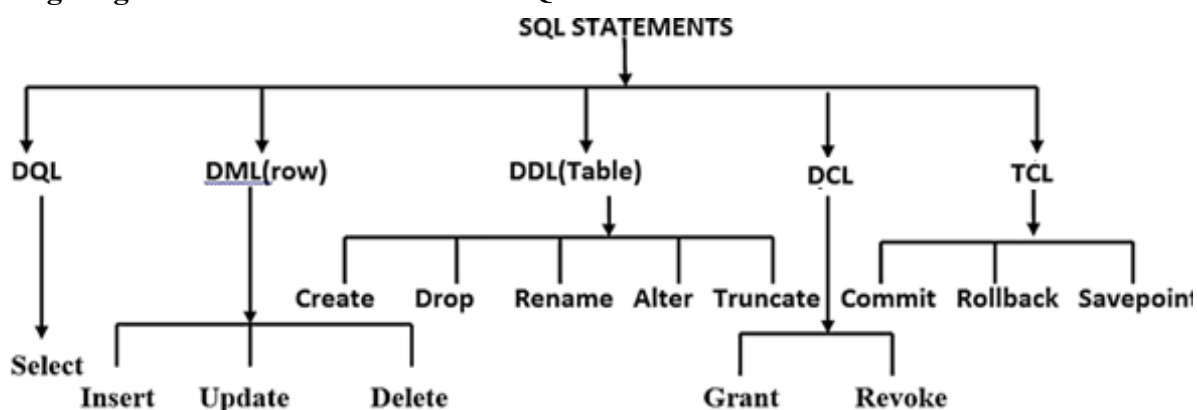
5. Savepoints for Partial Rollback:

- SAVEPOINT allows setting a **marker (savepoint)** within a transaction so you can roll back only some part of the transaction if needed.

Common TCL Commands in SQL are:

TCL Commands	Purpose
COMMIT	Saves all changes made during the transaction permanently
ROLLBACK	Undoes changes made during the transaction
SAVEPOINT	Sets a point in a transaction to roll back to selectively
SET TRANSACTION	Sets properties for the current transaction (like mode or isolation level)

Following diagram shows Classification of SQL commands:



Rules to use SQL: (Rules to use SQL commands):

There are following rules which are used while entering SQL commands-

- 1) SQL starts with a verb (i.e. a SQL action word).

Example: SELECT statements. This verb may have additional adjectives like FROM

- 2) Each verb is followed by number of clauses.

Example: FROM, WHERE, HAVING etc.

- 3) Each verb and clause are separated from each other by space.

Example: DROP TABLE EMP;

- 4) A comma (,) separates parameters.

Example: SELECT rollno, name FROM student;

- 5) SQL statement is terminated with a semicolon (;)

- 6) Reserved words cannot be used as identifiers (Name given by programmer for table, table field, function, variable etc.)

- 7) Identifiers can be combination of alphabets, digits or underscore _

- 8) Identifiers must start with alphabets or underscore. i.e. it does not start with digit.

- 9) Length of identifier is up to 30 characters long.

- 10) Special characters like # , %, @ etc. also spaces are not allowed in identifier.

- 11) Character and date constants (literals) must be enclosed within single quotes like '01-may-2001' or '10d'

SQL reserve words:

- To learn SQL, we have to know different reserve words available in it.
- The total number of SQL reserved words varies depending on the specific SQL standard (SQL version) or the particular database system. The most commonly used SQL reserve words are-

SQL Keyword	Description
SELECT	Used to retrieve data from a database.
FROM	Specifies the table(s) from which to retrieve data in a SELECT statement.
WHERE	Filters records based on a specified condition in SELECT, UPDATE, or DELETE statements.
INSERT	Used to add new rows of data into a table.
INTO	Used with INSERT to specify the target table.
VALUES	Used with INSERT to specify the data values to be inserted.
UPDATE	Used to modify existing data in a table.
SET	Used with UPDATE to specify the new values for columns.
DELETE	Used to remove rows from a table.
CREATE	Used to create new database objects like tables, views, indexes, or databases.
TABLE	Used with CREATE to define a new table.
ALTER	Used to modify the structure of an existing database object.
DROP	Used to remove database objects from the database.
AND	A logical operator used to combine multiple conditions in a WHERE clause.

OR	A logical operator used to combine multiple conditions in a WHERE clause.
NOT	A logical operator used to negate a condition.
NULL	Represents a missing or unknown value.
ORDER BY	Sorts the result set of a query.
GROUP BY	Groups rows that have the same values in specified columns into summary rows.
HAVING	Filters groups based on a specified condition, used with GROUP BY.
JOIN	Combines rows from two or more tables based on a related column between them.
AS	Used to assign an alias to a column or table.
DISTINCT	Used with SELECT to retrieve only unique values.
PRIMARY KEY	A constraint that uniquely identifies each record in a table.
FOREIGN KEY	A constraint that links two tables together.
CHECK	A constraint that limits the value range that can be placed in a column.
CONSTRAINT	Adds or deletes a constraint on a table.

SQL data types used in Oracle:

Oracle SQL provides a range of built-in data types to store various kinds of information.

The following table shows commonly SQL data types used in Oracle along with their descriptions and typical uses:

Data Type	Description	Typical Use												
VARCHAR2(size)	Variable-length character string up to size	Storing text strings of variable length												
CHAR(size)	Fixed-length character string of length size	Storing text strings of fixed length												
NUMBER(p,s)	Numeric data with precision p and scale s E.g. <table> <tr> <th>Definition</th><th>Allowed Values</th><th>Notes</th></tr> <tr> <td>NUMBER (5, 2)</td><td>Max 5 digits, 2 after decimal</td><td>123.45 valid, 1234.56 invalid (6 digits)</td></tr> <tr> <td>NUMBER (7, 0)</td><td>Max 7 digits, no decimal</td><td>1234567 valid, 12345678 invalid</td></tr> <tr> <td>NUMBER</td><td>No precision/scale specified</td><td>Any number within Oracle limits</td></tr> </table>	Definition	Allowed Values	Notes	NUMBER (5, 2)	Max 5 digits, 2 after decimal	123.45 valid, 1234.56 invalid (6 digits)	NUMBER (7, 0)	Max 7 digits, no decimal	1234567 valid, 12345678 invalid	NUMBER	No precision/scale specified	Any number within Oracle limits	Storing integers and decimals (financial data)
Definition	Allowed Values	Notes												
NUMBER (5, 2)	Max 5 digits, 2 after decimal	123.45 valid, 1234.56 invalid (6 digits)												
NUMBER (7, 0)	Max 7 digits, no decimal	1234567 valid, 12345678 invalid												
NUMBER	No precision/scale specified	Any number within Oracle limits												
FLOAT	Floating-point number	Storing approximate numeric values												
DATE	Date and time (year, month, day, hour, minute, second)	Storing dates and timestamps												

TIMESTAMP	Date and time with fractional seconds	Storing precise timestamps
CLOB	Character Large Object	Storing large text data
BLOB	Binary Large Object	Storing large binary data (images, audio files, and video files etc.)
RAW(size)	Raw binary data of length size	Storing binary data of fixed length
LONG	Variable-length character data	Legacy support for large text
NCHAR(size)	Fixed-length Unicode character string (making suitable for multi-language applications)	Storing fixed-length Unicode data
NVARCHAR2(size)	Variable-length Unicode character string (making suitable for multi-language applications)	Storing variable-length Unicode text
INTERVAL YEAR TO MONTH	Interval of years and months	Storing periods like '2 years 3 months'
INTERVAL DAY TO SECOND	Interval of days, hours, minutes, seconds	Storing periods like '10 days 5 hours'

Explanation of all DDL (Data Definition Language) Commands-

1) CREATE command-

- The **CREATE** command is used to create new database objects such as **tables**, **views**, **indexes**, or even entire **databases**.
- Most commonly, it is used to **create a new table** in a database.
- When creating a table, we have to define the table's name and specify its columns (attributes or fields) along with their data types and any constraints (like primary key, not null, unique, check etc.)

Syntax to create table:

```
CREATE TABLE table_name (
  column1    datatype    constraint,
  column2    datatype    constraint,
  column3    datatype    constraint,
  . . . . .
);
```

Example:

```
SQL> CREATE TABLE employee (
  employee_id NUMBER(5) PRIMARY KEY,
  first_name  VARCHAR2(50),
  last_name   VARCHAR2(50) NOT NULL,
  email       VARCHAR2(100) UNIQUE,
  hire_date   DATE,
  salary      NUMBER(8, 2) CHECK (salary > 0)
);
```

Explanation of Each Part of above query:

1. CREATE TABLE emplo (...)

- This creates a new table named employee.
- The table will have six columns (fields).
- You define the structure of the table inside the parentheses.

2. employee_id NUMBER(5) PRIMARY KEY

- employee_id is a column that stores numeric values, up to 5 digits.
- PRIMARY KEY means:
 - This column must have **unique** values (no duplicates).
 - It **cannot be NULL** (must always have a value).

- It uniquely identifies each row in the table.
- 3. **first_name VARCHAR2(50)**
 - first_name is a column for storing a string (text).
 - VARCHAR2(50) means it can store up to **50 characters**.
 - It is **optional** (can be NULL).
- 4. **last_name VARCHAR2(50) NOT NULL**
 - Stores the employee's last name.
 - Same as first_name, but:
 - NOT NULL makes it **mandatory**. You **must** enter a last name when inserting a row.
- 5. **email VARCHAR2(100) UNIQUE**
 - Stores email addresses.
 - VARCHAR2(100) allows up to 100 characters.
 - UNIQUE means:
 - All email values must be **different**.
 - If you try to insert a duplicate email, Oracle will return an error.
- 6. **hire_date DATE**
 - Stores the date the employee was hired.
 - DATE is a special Oracle data type that stores both **date and time** (although time is optional if you don't specify it).
- 7. **salary NUMBER(8, 2) CHECK (salary > 0)**
 - Stores the employee's salary.
 - NUMBER(8, 2) means:
 - Up to **8 digits in total**, with **2 digits after the decimal point**.
 - Example: 123456.78 is valid.
 - CHECK (salary > 0) adds a **constraint**:
 - It ensures that the salary must be **greater than 0**.
 - If someone tries to insert a salary of 0 or a negative number, the query will fail.

Note: After executing above table creation query, 'employee' table structure created in database.

Also, we can see the created table structure using 'desc' command as follow-

➤ **DESC (or DESCRIBE) Command in Oracle:**

- The DESC (short for **DESCRIBE**) command in Oracle is used to display the **structure of a table** — i.e., the list of its columns along with their data types, nullability, and other attributes.
- When you run this command, Oracle returns information like:

Name	Null?	Type
Column name	NULL	Data type(Size)

Example-

```
SQL> desc employee;
Name                               Null?    Type
-----
EMPLOYEE_ID                       NOT NULL NUMBER(5)
FIRST_NAME                        VARCHAR2(50)
LAST_NAME                         NOT NULL VARCHAR2(50)
EMAIL                             VARCHAR2(100)
HIRE_DATE                         DATE
SALARY                            NUMBER(8,2)
```

Creating a Table from Previously Created Table:

We can also able to create a new table from previously created table.

Syntax:

SQL> create table TargetTableName as select * from SourceTableName;

- This query creates a **new table** (TargetTableName) and **copies the structure and data** from an existing table (SourceTableName).
- This is often called a **CTAS** operation (Create Table As Select).

Example:

SQL> create table employeeDemo as select * from employee;

- This command creates a new table named **employeeDemo** by **copying the structure** (column names and data types) and all data (rows) from an existing table called

employee.

🔍 Detailed Explanation of above query:

◊ What is copied:

- ☒ Column names
- ☒ Column data types
- ☒ All rows from the employee table

◊ What is not copied:

- ☒ Constraints like PRIMARY KEY, NOT NULL, UNIQUE, CHECK
- ☒ Indexes
- ☒ Triggers
- ☒ Sequences or auto-increment settings
- ☒ Default values

So, the new table **employeeDemo** will have the same data and structure **but without constraints or special properties**.

Creating a Table from Previously Created Table by taking appropriate fields of existing table also change the column name at table creation:

```
SQL> create table TargetTableName(newColName1,newColName2,...) as select  
oldColName1,oldColName2,... from SourceTableName;
```

Example:

```
SQL>create table empDemo1(empno,fname,lname) as select employee_id,first_name,last_name from  
employee;
```

➤ Explanation of above query:

This query creates a **new table** called **empDemo1** by selecting specific columns (employee_id, first_name, last_name) from the existing **employee** table. It also **renames** the columns in the new table having names **empno,fname,lname** respectively.

SQL CONSTRAINTS USED IN ORACLE: (DATA INTEGRITY IN ORACLE)

- We know that, at the time of table creation we specify the constraints for the column of table.
- SQL constraints are the rules that are applied to columns in a database table to enforce data integrity.
- SQL constraints ensure the accuracy of the data, and limit the type of data that can be inserted or updated. Because of constraints, correct data will be inserted into database table.

- Constraints can be divided into two types as follow:

1) Column-level Constraint- The constraints can be specified immediately after the column definition is called “column-level constraint”.

Example- SQL> create table emp(eid number **primary key**,ename char(10));

In above example, primary key constraint is specified to eid column immediately hence it is column-level constraint.

2) Table-level Constraint- The constraints can be specified after all columns and at the end of table called “table-level constraint”.

E.g.- SQL>create table emp(eid number,ename char(10), **CONSTRAINT pkey PRIMARY KEY(eid)**);

In above example, primary key constraint is specified to eid column at the end of table creation hence it is table-level constraint.

- Below are the most commonly SQL constraints used in oracle:

Constraint	Description
NOT NULL	Column cannot have NULL values
UNIQUE	All values in the column must be unique.

CHECK	Value must satisfy a condition
DEFAULT	Sets a default value when none is provided during insertion
PRIMARY KEY	Uniquely identifies each row (NOT NULL + UNIQUE)
FOREIGN KEY	Enforces referential integrity between tables

- We explain all these constraints with example as follow-

1. NOT NULL Constraint:

- In Oracle Database, the **NOT NULL** constraint is used to ensure that a column **cannot store NULL values**. It is one of the most basic **integrity constraints** and is typically used when a column **must always have a value** for every row in a table.
- NOT NULL ensures that the column must have a value i.e. it cannot be left blank (NULL) during an INSERT or UPDATE.
- NOT NULL constraint **defined only at the column level, not at the table level**.

✅ Example:

```
SQL> CREATE TABLE Student (
    StudentID NUMBER,
    Name VARCHAR2(50) NOT NULL);
```

❌ Invalid Insert:

```
SQL> INSERT INTO Student (StudentID) VALUES (1);
-- ORA-01400: cannot insert NULL into ("STUDENT"."NAME")
```

2. UNIQUE Constraint:

- In Oracle, the **UNIQUE constraint** ensures that **all values in a column (or combination of columns) are distinct** (i.e. **unique**.) — no two rows can have the same value(s) in the specified column(s).
- The **UNIQUE constraint** prevents **duplicate values** in a column or group of columns but **allows NULLs** (because NULL is not considered equal to any other NULL in SQL Oracle).
- UNIQUE constraint can be defined at both Column-level and also Table-level.
- A single table can have multiple UNIQUE constraints on different columns or combinations.
- UNIQUE constraint can be applied to multiple columns together (composite unique constraint).
- Basically, UNIQUE key constraint used for maintaining **data integrity** in a relational database.

✅ Example:

```
SQL> CREATE TABLE UserLogin (
    UserID NUMBER,
    Email VARCHAR2(100) UNIQUE);
```

❌ Invalid Insert:

```
SQL> INSERT INTO UserLogin VALUES (1, 'user@example.com');
SQL> INSERT INTO UserLogin VALUES (2, 'user@example.com');
-- ORA-00001: unique constraint violated
```

3. CHECK Constraint:

- In Oracle, a **CHECK constraint** is used to enforce **domain integrity** by limiting the values that can be placed in a column.
- CHECK constraint ensures that data entered into a column **satisfies a logical condition**.
- The **CHECK constraint** specifies a **condition that must be true** for each row in the table.
- If the condition evaluates to TRUE then only INSERT or UPDATE operation carried out.
- If the condition evaluates to FALSE, Oracle will reject the INSERT or UPDATE operation.
- CHECK constraint can be defined at column-level or Table-level also.

✅ Example:

```
SQL> CREATE TABLE Account (
    AccountID NUMBER,
    Balance NUMBER CHECK (Balance > 0));
```

❌ Invalid Insert:

```
SQL>INSERT INTO Account VALUES (1, -100);  
-- ORA-02290: check constraint violated
```

4. DEFAULT Constraint:

- In Oracle, a **DEFAULT constraint** is used to **automatically assign a value to a column** when no value is explicitly provided during an INSERT operation.
- It helps enforce **default business rules** and reduces the need for manual value insertion.
- It Sets a default value for a cell if no value is given while inserting a record in table.
- It is typically used at the column level.
- Oracle doesn't support DEFAULT constraint at table-level.

✓ Example:

```
SQL>CREATE TABLE Orders (  
    OrderID NUMBER,  
    OrderDate DATE DEFAULT SYSDATE);  
SQL> INSERT INTO Orders (OrderID) VALUES (1001);  
Here, OrderDate will be set to current date (SYSDATE)  
Check by result by using SELECT statement.
```

5. PRIMARY KEY Constraint:

- Basically, primary key is used to identify a row uniquely i.e. it avoids insertion of duplicate records in table.
- Primary key is a combination of **UNIQUE** and **NOT NULL**
- In Oracle, a PRIMARY KEY constraint is used to uniquely identify each row in a table.
- It is one of the most important constraints in relational databases and enforces both uniqueness and not null rules on the column(s) it is applied to.
- A PRIMARY KEY constraint ensures that:
 - No two rows have the same value(s) in the primary key column(s).
 - The column(s) cannot be NULL.
- A table can only have one primary key, but a primary key can consist of multiple columns called “composite key primary key”.
- A primary key constraint can be applied at column level and table level also.

✓ Example:

```
SQL>CREATE TABLE Department (  
    DeptID NUMBER PRIMARY KEY,  
    DeptName VARCHAR2(50));  
Equivalent syntax for above query with named constraint:
```

```
SQL>CREATE TABLE Department (  
    DeptID NUMBER,  
    DeptName VARCHAR2(50),  
    CONSTRAINT pk_dept PRIMARY KEY (DeptID));
```

Executing following insert record operation on above table-

```
SQL> insert into Department values(1, 'Science');  
1 row created.
```

```
SQL> insert into Department values(2, 'Arts');  
1 row created.
```

✗ Invalid Insert:

```
SQL> insert into Department values(2, 'Commerce');  
ERROR at line 1:  
ORA-00001: unique constraint (SYSTEM.SYS_C004237) violated  
SQL> insert into Department (DeptName) values('Commerce');  
ERROR at line 1:
```

ORA-01400: cannot insert NULL into ("SYSTEM"."DEPARTMENT"."DEPTID")

6. FOREIGN KEY Constraint:

- In Oracle, a FOREIGN KEY constraint (also known as a referential integrity constraint) ensures that the values in one table match values in another table's primary or unique key.
- It establishes a parent-child relationship between two tables.
- A foreign key links one or more columns (the "child" table) to the primary key or unique key in another table (the "parent" table).
- It ensures that you cannot insert or update a value in the child table unless it exists in the parent table.
- Basically, Foreign key is used to create a relationship between tables in database.
- A FOREIGN key constraint can be applied at column level and table level also.

✓ Parent Table:

```
SQL> CREATE TABLE Department (  
    DeptID NUMBER PRIMARY KEY,  
    DeptName VARCHAR2(50));
```

Executing following insert record operation on above table-

```
SQL> insert into Department values(1, 'Science');
```

1 row created.

```
SQL> insert into Department values(2, 'Arts');
```

1 row created.

✓ Child Table with Foreign Key:

(Column Level foreign Key)

```
SQL>CREATE TABLE Employee(  
EmpID    NUMBER PRIMARY KEY,  
EmpName  VARCHAR2(50),  
DeptId   NUMBER REFERENCES  Department(DeptID));
```

(Table Level foreign Key)

```
SQL>CREATE TABLE Employee (  
    EmpID NUMBER PRIMARY KEY,  
    EmpName VARCHAR2(50),  
    DeptID NUMBER, FOREIGN KEY (DeptID) REFERENCES Department(DeptID));
```

✓ Valid Insert:

```
SQL>INSERT INTO Employee VALUES (101, 'Ravi', 1);
```

1 row created.

✗ Invalid Insert:

```
SQL>INSERT INTO Employee VALUES (102, 'Kiran', 99);
```

-- ORA-02291: integrity constraint violated – parent key not found

2) ALTER command:

- The ALTER command in Oracle SQL is DDL command used to **modify the structure of an existing table**.
- ALTER command allows you to:
 - Add, modify (Change data type or size of column) or drop table columns.
 - Rename columns name or the table name.
 - Add or drop constraints.
 - Set or drop default values.

Let's see ALTER command in details-

2.1) Adding Columns in table-

We can add new columns into existing table using alter command as-

Syntax-

```
SQL>ALTER TABLE tableName ADD(newCol1 datatype,newCol2 datatype,newCol3 datatype,...);
```

The above query adds **multiple new columns** newCol1,newCol2,newCol3 to an existing table.

✔ **Example:**

SQL>ALTER TABLE Department **ADD**(DeptAddress VARCHAR2(50));

The above query adds **single column** DeptAddress to an existing table Department.

SQL>ALTER TABLE Employee **ADD**(eadd VARCHAR2(10),contact number(10));

The above query adds **multiple columns** eadd and contact to an existing table Employee.

2.2) Modify Column (Data Type or Size):

We can modify existing columns data type and also its size using alter command as-

Syntax:

SQL>ALTER TABLE tableName **MODIFY**(colName newdatatype(newSize));

The above command changes the data type and size of an existing column in a table.

✔ **Example:**

SQL>ALTER TABLE Employee **MODIFY**(eName CHAR(150));

The above command changes the data type and size of eName column in Employee table.

❖ Restrictions on Altering Columns:

Note the following important points while altering the column-

1. **Cannot directly change column order** in a table.
2. **Cannot decrease column size** if table having existing data that doesn't fit to the new size.
3. **Cannot change datatype** if it's incompatible with existing data.
4. **Cannot change datatype of a column used in a primary/foreign key** unless you drop the constraint first.
5. **Renaming a column** may break dependent objects (other table, views, triggers, PL/SQL code) unless they are updated.

2.3) Drop Column:

We can drop existing column from table using alter command as-

➤ Dropping single column:

Syntax:

SQL>ALTER TABLE tableName **DROP COLUMN** colName;

The above command drops single column **colName** from existing table.

✔ **Example:**

SQL>ALTER TABLE Employee **DROP COLUMN** email;

The above command drops single column **email** from existing table Employee.

➤ Dropping multiple columns:

Syntax:

SQL>ALTER TABLE tableName **DROP** (colName1,colName2,...);

The above command drops multiple columns **colName1, colName2** from existing table.

✔ **Example:**

SQL> ALTER TABLE Employee **DROP** (addr1, phone);

The above command drops multiple columns **addr1,phone** from existing table Employee.

❖ Restrictions on Dropping Columns:

Note the following important points while dropping the column-

1. **Cannot drop all columns** — at least one column must remain in table.
2. **Cannot drop a column** that is part of:
 - A primary key (unless you drop the PK constraint first).
 - A unique constraint.
 - A foreign key reference (unless constraint is dropped or disabled).
3. Dropping a column **permanently deletes data** (no undo except backup).
4. Large table column drops may require significant time and disk space internally

2.4) Rename columns name or the table name:

Alter command also used for renaming the column name and table name also. We explain it as follow-

2.4.1) Renaming a Column:

Syntax:

SQL> ALTER TABLE table_name RENAME COLUMN old_column_name TO new_column_name;

Example:

SQL> ALTER TABLE emp RENAME COLUMN name TO ename;

The above query renames 'name' column to 'ename' column of table 'emp'.

❖ Points To remember while renaming the column:

- The new column name must be unique in the table.
- Renaming a column does **not** change data in it.
- May break dependent objects (views, triggers, PL/SQL code) that refer to the old column name — you must update them manually.

2.4.2) Renaming a Table:

Syntax:

SQL> ALTER TABLE old_table_name RENAME TO new_table_name;

Example:

SQL> ALTER TABLE emp RENAME TO empTbl;

The above query renames 'emp' table to 'empTbl'.

2.5) Add or drop constraints:

We can also add or drop constraints using alter command, which is explained as follow-

2.5.1) Adding Constraints with ALTER:

We can add constraints to an existing table without recreating it.

Syntax:

SQL> ALTER TABLE table_name ADD CONSTRAINT constraint_name constraint_type (col_name);

Here,

Constraint_name is Name given by user for constraint which is an identifier.

Constraint Types is PRIMARY KEY, UNIQUE, FOREIGN KEY, CHECK, NOT NULL (special case — handled via MODIFY)

Example-

SQL> ALTER TABLE empTbl ADD CONSTRAINT ukey unique (salary);

The above query add unique constraint for field 'salary' of 'empTbl' table.

2.5.2) Dropping Constraints with ALTER:

We can also drop constraints from an existing table.

Syntax:

SQL> ALTER TABLE table_name DROP CONSTRAINT constraint_name [CASCADE];

Here,

constraint_name is name of constraint that we are going to drop.

CASCADE is optional used when other constraints depend on the one you're dropping (mostly for PRIMARY KEY that has child foreign keys).

Note that: Dropping a constraint does **not** delete the column.

Example-

SQL> ALTER TABLE empTbl DROP CONSTRAINT ukey;

The above query, drops the constraint 'ukey' that we had previously added.

2.6) Set or drop default values using ALTER command:

2.6.1) Setting a Default Value using ALTER:

We can assign a **default value** to a column so that when an INSERT statement omits that column, Oracle automatically assigns the default value for respective column.

Syntax:

SQL> ALTER TABLE table_name MODIFY column_name DEFAULT default_value;

Example:

```
SQL> ALTER TABLE empTbl MODIFY ename default 'AJAY';
```

After running above query, table is altered successfully.

Note: if you **insert a new row** into empTbl **without providing** a value for ename, Oracle will **automatically insert 'AJAY'** in that column.

2.6.2) Dropping a Default Value using ALTER:

To remove or drop an assigned default value, we have to set it to NULL.

Syntax:

```
SQL> ALTER TABLE table_name MODIFY column_name DEFAULT NULL;
```

After running above query, table is altered successfully.

Example:

```
SQL> ALTER TABLE empTbl MODIFY ename DEFAULT NULL;
```

The above query, drops assigned default value 'JIBRAIL' for 'ename' of table empTbl.

3) DROP command:

- This command also belongs to DDL category which is used to Delete database objects such as tables, views, index, sequence, user etc. permanently.
- Once an object is dropped, all the data and structure associated with it are deleted and **cannot be recovered** unless you have a backup or recycle bin is enabled.
- Syntax:

```
SQL> DROP object_type object_name;
```

Here:

- object_type can be TABLE, VIEW, INDEX, SEQUENCE, etc.
- object_name is the name of the object to be dropped.

Example 1: Dropping a table-

```
SQL> DROP TABLE employee;
```

- This command permanently removes the employee table and all its data from the database.
- If the **Recycle Bin** is enabled (in Oracle 10g and later), the table (which was created on **users** **tablespace**) can be recovered using FLASHBACK command.
- By default, all tables are created in 'system tablespace' but we can create table in 'users tablespace' as follow-

```
CREATE TABLE table_name (  
    column1 datatype constraint,  
    column2 datatype constraint,  
    -- ... more columns  
)TABLESPACE tablespaceName;
```

- Example -

```
SQL> CREATE TABLE emp101(eid number) TABLESPACE users;
```

Above query creates 'emp101' table in **users** tablespace.

➤ Check parameter for the recyclebin whether it is ON or OFF:

```
SQL> show parameter recyclebin;
```

This would produce output as follow-

NAME	TYPE	VALUE
recyclebin	string	on

If it is OFF then, To enable or ON the recyclebin at the system level:

Open a SQL*Plus session or a similar tool and connect as a user with ALTER SYSTEM privileges (e.g., SYSDBA) and execute the following command

```
SQL> ALTER SYSTEM SET recyclebin = ON;
```

➤ To view the content of recyclebin:

```
SQL> show recyclebin;
```


This would produce list of deleted tables (only **users** **tablespace**). As shown bellow-

```
SQL> show recyclebin;
ORIGINAL NAME      RECYCLEBIN NAME      OBJECT TYPE  DROP TIME
-----
EMP101             BIN$zi4E8ESCSKWZL07u6PHRKQ==$0  TABLE      2025-08-16:13:44:07
```

➤ Viewing data from dropped table using 'Recyclebin name':

```
SQL> Select * from "RecyclebinName"
```

Example:

```
SQL> Select * from "BIN$zi4E8ESCSKWZL07u6PHRKQ==$0";
```

This would produce output:

```
SQL> select *from "BIN$zi4E8ESCSKWZL07u6PHRKQ==$0";

      EID
-----
        1
        2
        3
```

➤ To recover dropped table from recyclebin:

```
SQL> FLASHBACK TABLE emp101 TO BEFORE DROP;
```

The above query recycles back the dropped table.

Note:

- The DROP command is **irreversible** unless you are using features like **Flashback** or **Recycle Bin**.
- **Important:** Only those dropped table **shown** in recyclebin which are created on **users tablespace**.
- **Important:** Those dropped table are **not shown** in recyclebin which are created on **system tablespace**.
- It requires appropriate **privileges**. For example, dropping a table requires either ownership or DROP ANY TABLE privilege.
- ◆ **Example 2: Dropping a View**

```
SQL>DROP VIEW emp_view;
```

 - The above command deletes the view 'emp_view'.
 - No actual data is deleted, since views are virtual tables based on queries.
- ◆ **Example 3: Dropping a Sequence**

```
SQL>DROP SEQUENCE emp_seq;
```

 - Removes the sequence 'emp_seq' used for generating numbers.
- ◆ **Example 4: Dropping a User (and their schema)**

```
SQL>DROP USER hr CASCADE;
```

 - Removes the user 'hr' and **all objects** (tables, views, etc.) owned by that user.
 - CASCADE is required to drop a user along with all dependent objects.

4) TRUNCATE command:

- This is also DDL command used to delete all rows i.e. all tuples i.e. all records from a table **quickly and permanently, without logging individual row deletions**. (faster than DELETE)
- It is faster and less resource-intensive than the DELETE command because it doesn't generate as much undo and redo data.

Syntax:

```
SQL> TRUNCATE TABLE table_name;
```

Example- Let's say we have a table employees:

```
SQL>CREATE TABLE employees (
        emp_id      NUMBER,
        emp_name     VARCHAR2(100));
```

Insert some data:

```
SQL>INSERT INTO employees VALUES (1, 'AJAY');
```

```
SQL>INSERT INTO employees VALUES (2, 'VIJAY');
```

```
SQL>COMMIT;
```

Now, truncate the table:

```
SQL>TRUNCATE TABLE employees;
```

✓ This will:

- Remove all rows from employees.
- Free up the space used by the data.
- Not fire any DELETE triggers (if any exist).
- Not be rolled back (it's a DDL operation which is auto-commit).

Difference between TRUNCATE and DELETE command:

TRUNCATE	DELETE
1. TRUNCATE command belongs DDL category.	1. DELETE command belongs to DML category.
2. TRUNCATE command not rolled back, since it is DDL	2. DELETE command rolled back, since it is DML
3. WHERE clause is Not supported in TRUNCATE	3. WHERE clause is supported in DELETE
4. TRUNCATE command not fires DELETE triggers to delete the record.	4. DELETE command fires DELETE triggers to delete the record.
5. Working speed of TRUNCATE is Faster than DELETE because it deletes <u>all records at one time</u> and free the space.	5. Working speed of DELETE is Slower than TRUNCATE because it <u>deletes record one by one</u> .
6. Deletes all records i.e. it does not delete specific row (Since, WHERE clause is not supported)	6. It deletes specific row (record) with WHERE clause

4) RENAME command:

This is also DDL command used to Rename a database object such as tables, views, index, sequence etc.

Syntax:

```
SQL>RENAME old_name TO new_name;
```

Here,

old_name is Current name of the object.

new_name is New name you want to assign.

Example:

```
SQL>RENAME employees TO staff;
```

Above query renames 'employees' to new name 'staff'

➤ Important points to be noticed while performing RENAME command-

1. You can only rename **your own** objects.
2. You **cannot** rename **columns** using RENAME. Use ALTER TABLE for that.
3. You **cannot** rename objects in another schema.

Explanation of all DML (Data Manipulation Language) Commands-

1) INSERT command-

- In Oracle (and SQL in general), the INSERT command is used to add new records (rows) into a table.
- When inserting a single row (record) into the table, the insert operation:
 - First creates a new empty row in table.
 - Then insert statement loads passed values to the specified columns of table.
- In INSERT INTO statement, table columns and values have one to one relationship. i.e. First value inserted into first column, second value inserted into second column and so on.
- There are **different forms** of the INSERT statement which are explained as follow-

1) Insert a single row (record) by specifying column name :

Syntax:

```
SQL>INSERT INTO table_name (column1, column2, ..., columnN) VALUES (value1, value2, ..., valueN);
```

Suppose you have a table called employees:

```
SQL>CREATE TABLE employees (
    emp_id    NUMBER,
    first_name VARCHAR2(50),
    last_name  VARCHAR2(50),
    salary    NUMBER);
```

You can insert a single row like this:

```
SQL> INSERT INTO employees (emp_id, first_name, last_name, salary) VALUES (101, 'AMIT', 'ROY', 50000);
```

2. Insert a single row (record) without specifying column names (all columns):

Syntax:

```
SQL> INSERT INTO table_name VALUES (value1, value2, ..., valueN);
```

Example:

```
SQL> INSERT INTO employees VALUES (102, 'JITENDRA', 'KALE', 60000);
```

Note: The values must match the **order and data type** of the table's columns.

3. Insert multiple rows (using INSERT ALL):

Oracle allows multiple inserts in one statement using INSERT ALL:

Syntax:

```
SQL> INSERT ALL
      INTO table_name (col1, col2, col3, ....) VALUES (val1, val2, val3, ...)
      INTO table_name (col1, col2, col3, ....) VALUES (val1, val2, val3, ...)
      SELECT * FROM dual;
```

Example-

```
SQL> INSERT ALL
      INTO employees (emp_id, first_name, last_name, salary) VALUES (103, 'Alice', 'Brown', 55000)
      INTO employees (emp_id, first_name, last_name, salary) VALUES (104, 'Bob', 'White', 52000)
      SELECT * FROM dual;
```

Note:

- Here, DUAL is a special one-row table in Oracle often used in insert/select operations.

4. Insert data from one table to another table:

- This is useful for copying data from one table to another.

Syntax:

```
SQL> INSERT INTO Dest_table (col1, col2, col3, ....)
      SELECT col1, col2, col3, .... FROM Source_table;
```

Example-

```
SQL> INSERT INTO empDest (emp_id, first_name, last_name, salary)
      SELECT emp_id, first_name, last_name, salary FROM employees;
```

- The above query **copies data** from source table **employees** into another destination table **empDest**.
- **Note:** While inserting data from one table to another table, destination table should be created first.

➤ Inserting values in table using Substitution Variables (&variable):

- **&** is used in tools like **SQL (ORACLE)** to prompt the user for input. (**interactive data entry**)
- When the query is run, Oracle will **ask the user to enter a value** for each variable.

Syntax:

```
SQL> INSERT INTO table_name VALUES (&Col1, &Col2, ..., &ColN);
```

In above syntax, **&** is used along with column of table will prompt user for input.

Example:

```
SQL> INSERT INTO student VALUES (&rollno, '&name', '&address');
Enter value for rollno: 101
Enter value for name: RAJESH
Enter value for address: PUNE
old 1: INSERT INTO student VALUES (&rollno, '&name', '&address')
new 1: INSERT INTO student VALUES (101, 'RAJESH', 'PUNE')

1 row created.
```

Note: Use single quote for column having data type varchar2 or char or date

Use of forward slash / on SQL terminal:

- / is used to **execute the most recently entered SQL statement or PL/SQL block**.
- That is forward slash / executes previously executed SQL command or PL/SQL block.

Example-

```
SQL> /
Enter value for rollno: 102
Enter value for name: DINESH
Enter value for address: SANGOLA
old 1: INSERT INTO student VALUES(&rollno, '&name', '&address')
new 1: INSERT INTO student VALUES(102, 'DINESH', 'SANGOLA')

1 row created.
```

In above example, we use forward slash / to execute insert command again.

2) SELECT command-

- The **SELECT command in Oracle** is used to **retrieve data** from one or more tables.
- **Syntax:**

SQL>SELECT column1, column2,,columnN **FROM** table_name **WHERE** condition;

Here, column1, column2,,columnN are the table columns which will retrieve.

table_name is name of the table from which data is retrieved.

WHERE is clause which is optional used when specific data would be retrieved as per condition.

Example-

```
SQL> SELECT name, address FROM student WHERE rollno=102;

NAME                ADDRESS
-----
DINESH              SANGOLA
```

Here, above query fetches name and address of student having rollno is 102 from student table.

Different examples of SELECT statement in Oracle:

1. Select all columns:

Syntax: **SQL>SELECT *** **FROM** table_name;

Example-

SQL>SELECT * **FROM** student;

👉 This shows **all rows and all columns** from the student table.

Output:

```
SQL> SELECT * FROM student;

ROLLNO NAME                ADDRESS
-----
101 RAJESH                PUNE
102 DINESH                SANGOLA
```

2. Select specific columns (Project operations):

Syntax:

SQL>SELECT col1,col2,col3,... **FROM** table_name;

Example-

SQL>SELECT name, address **FROM** student;

👉 This shows only the name and address of each student.

Output:

```
SQL> SELECT name, address FROM student;

NAME          ADDRESS
-----
RAJESH        PUNE
DINESH        SANGOLA
```

3. Using WHERE clause to retrieve specific records: (Filter)

Syntax:

```
SQL>SELECT col1,col2,col3,... FROM table_name WHERE condition;
```

Example:

```
SQL>SELECT name, address FROM student WHERE rollno=102;
```

👉 Shows student name and address whose rollno is 102.

Output:

```
SQL> SELECT name,address FROM student WHERE rollno=102;

NAME          ADDRESS
-----
DINESH        SANGOLA
```

4. Sorting with ORDER BY:

Syntax:

```
SQL>SELECT col1,col2,col3,... FROM table_name ORDER BY col_name order;
```

Here,

order may be **DESC** or **ASC**.

- To sort data in descending order use DESC.
- To sort data in ascending order use ASC.

Example-

```
SQL>SELECT * FROM student ORDER BY rollno DESC;
```

👉 Shows all student record in **descending order of rollno**.

Output:

```
SQL> SELECT * FROM student ORDER BY rollno DESC;

ROLLNO NAME          ADDRESS
-----
102 DINESH        SANGOLA
101 RAJESH        PUNE
```

5. Using aliases (alternative name):

Syntax:

```
SQL>SELECT col1 AS newCol1,col2 AS newCol2,... FROM table_name;
```

Example-

```
SQL>SELECT rollno AS Roll,name AS FirstName, address AS Add1 FROM student;
```

👉 Renames the column headings in the output.

Output:

```
SQL> SELECT rollno AS Roll,name AS FirstName, address AS Add1 FROM student;

ROLL FIRSTNAME      ADD1
-----
101 RAJESH        PUNE
102 DINESH        SANGOLA
```

6. Using functions:

```
SQL>SELECT COUNT(*) AS Total_student FROM student;
```

👉 Shows the **total number of students**. (counts total number student)

Output:

```
SQL> SELECT    COUNT(*) AS Total_student FROM student;

TOTAL_STUDENT
-----
                2
```

***Note:** You can (SELECT) filter records with WHERE, sort with ORDER BY, rename with AS, and use functions like COUNT, SUM, AVG, etc.*

7. Eliminating Duplicate rows (records):

- In a table, there is possibility to exist a duplicate value. And if we want to filter or retrieve only one value from all duplicate values then DISTINCT clause is used along with SELECT.
- In short, DISTINCT clause is used to retrieve only unique data from table.
- The functionality of UNIQUE and DISTINCT clause are same.
- **DISTINCT clause (UNIQUE clause)-** Allows to remove duplicates data from the result set
- This clause only used with SELECT statement. It scans all the values of the specified columns and display only unique values. It eliminates rows that have exactly same contents in each column.

Syntax:

```
SQL>SELECT  DISTINCT  Col1,Col2,.....,ColN  From  tablename;
```

OR

```
SQL>SELECT  DISTINCT  *  from  tablename;
```

Example:

Suppose we have following 'student' table-

```
SQL> select * from student;

  ROLLNO  NAME      ADDRESS
-----
    101  RAJESH      PUNE
    102  DINESH      SANGOLA
    103  KARTIK      THATNE
    104  DINESH      SOLAPUR
    104  DINESH      SOLAPUR
```

The above table have duplicate records of DINESH. We can retrieve **all distinct records** from student table as follow-

```
SQL> select distinct * from student;

  ROLLNO  NAME      ADDRESS
-----
    102  DINESH      SANGOLA
    104  DINESH      SOLAPUR
    103  KARTIK      THATNE
    101  RAJESH      PUNE
```

Also, We can retrieve all **distinct name** from student table as follow-

```
SQL> select distinct name from student;

NAME
-----
RAJESH
KARTIK
DINESH
```

3) DELETE command-

- The **DELETE** command in Oracle is used to **delete or remove all rows or specific rows from a table**

based on a specified condition.

- **Syntax:**

SQL>DELETE FROM table_name WHERE condition;

Here,

table_name: The name of the table from which we want to delete data.

condition: The criteria to determine which rows should be deleted.

WHERE is clause used to give condition.

Example 1: Delete specific rows:

Syntax:

SQL>DELETE FROM table_name WHERE condition;

Suppose, we have a table called employee as follows:

```
SQL> select * from employee;
```

EMP_ID	EMP_NAME	DEPARTM
1	Ajay	HR
2	Vijay	IT
3	Rajesh	IT
4	Kiran	Finance

Now, we want to delete all employees from the IT department then its query will be-

SQL>DELETE FROM employee WHERE department = 'IT';

The above query deletes 2 records having department = 'IT'

The resultant employee table is-

```
SQL> select * from employee;
```

EMP_ID	EMP_NAME	DEPARTM
1	Ajay	HR
4	Kiran	Finance

Example 2: Delete all rows:

This will remove **all data** from the table, but the table still exists and remains empty.

Syntax:

SQL>DELETE FROM table_name;

```
SQL> delete from employee;
```

2 rows deleted.

This will remove **all data** from the employees table, but the table still exists and is empty.

❖ **Important Notes regarding DELETE command:**

- If you **omit the WHERE clause**, all rows in the table will be deleted, but the **table structure remains** as it is in database.
- The DELETE command can be **rolled back** if it hasn't been committed. (Since, it belongs to DML category)
- DELETE affects **row-level locks** and fires **DELETE triggers**, if any exist.

4) UPDATE command-

- The UPDATE command in Oracle is used to **modify existing data** stored in a table.

- **Syntax:**

SQL>UPDATE table_name SET column1 = value1, column2 = value2, ... WHERE condition;

Here,

table_name: Name of the table where the update will happen.

SET: Is Keyword used to specify which columns to update and with their new values.

WHERE is clause used to give condition.

condition: The criteria to determine which rows should be updated.

Note: **Without a WHERE clause, all rows will be updated.**

Suppose, we have a table called employee as follows:

```
SQL> select * from employee;
```

EMP_ID	EMP_NAME	DEPARTM
1	Ajay	HR
2	Vijay	IT
3	Rajesh	IT
4	Kiran	Finance

Example1. Update a Specific Row

Suppose we want to change **Vijay's** department from IT to HR:

```
SQL> UPDATE employee SET department = 'HR' WHERE emp_name = 'Vijay';
```

1 row updated.

```
SQL> select * from employee;
```

EMP_ID	EMP_NAME	DEPARTM
1	Ajay	HR
3	Rajesh	IT
2	Vijay	HR
4	Kiran	Finance

✓ This update only Vijay's record.

2. Update Multiple Columns:

Let's say we want to change **Rajesh's** department to Finance and rename him to **Jitendra**:

```
SQL> UPDATE employee SET department = 'Finance', emp_name='Jitendra' WHERE emp_id = 3;
```

1 row updated.

```
SQL> select * from employee;
```

EMP_ID	EMP_NAME	DEPARTM
1	Ajay	HR
2	Vijay	HR
3	Jitendra	Finance
4	Kiran	Finance

✓ This update only emp_id = 3 record.

3. Update All Rows:

If we do not include a WHERE clause in UPDATE command then it updates all rows.

```
SQL> update employee SET department='General';
```

4 rows updated.

```
SQL> select * from employee;
```

EMP_ID	EMP_NAME	DEPARTM
1	Ajay	General
2	Vijay	General
3	Jitendra	General
4	Kiran	General

✓ This changes every employee's department to General.

Explanation of all TCL (Transaction Control Language) Commands-

- We know that, all DML commands are **not auto-commit** therefore the result of DML commands are **not save permanently** in database.
- TCL commands are used to manage the changes made in the database by a DML statements called as a 'transaction'. That is, TCL commands are used to manage transactions in database.

Let's see different TCL commands in detail-

1) COMMIT and ROLLBACK command-

- The **COMMIT** command in Oracle is used to **save all changes made during the current transaction permanently** to the database.
- When **COMMIT** is fired then all transactions (insert,update,delete) is **saved permanently into database** and **transaction becomes completed**.
- **Syntax:**
SQL>COMMIT;
- The **ROLLBACK** command in Oracle is used to **undoes all changes since last commit**.
- **Syntax:**
SQL>ROLLBACK;
- **Note:** A **transaction** is a sequence of one or more DML operations (like INSERT, UPDATE, DELETE) performed as a single logical unit of work. These changes are **temporary** until you explicitly save them using COMMIT.
- Until you run COMMIT, you can **undo** changes with ROLLBACK.
- Once you COMMIT, changes become permanent in database and visible to other users.

Example-Suppose, we have 'employee' table with few records as follow-

```
SQL> select * from employee;
```

EMP_ID	EMP_NAME	DEPARTM
1	Ajay	General
2	Vijay	General
3	Jitendra	General
4	Kiran	General

Scenario: You want to update **Kiran's** department from 'General' to 'Finance' and delete **Ajay's** record:
Then we execute following SQL queries-

SQL>UPDATE employee SET department = 'Finance' WHERE name = 'Kiran';

SQL>DELETE FROM employee WHERE name = 'Ajay';

At this point, these changes are **not yet permanent**.

- **If you run:**

SQL>ROLLBACK;

The changes will be undone. Kiran's department stays 'General', and 'Ajay' remains in the table.

- **But if you want to save the changes permanently, run:**

SQL>COMMIT;

Now, Kiran's department is updated to Finance, and Ajay's record is permanently deleted from table.

2) SAVEPOINT command:

- In Oracle SQL, the **SAVEPOINT** command is used to set a *marker* (a checkpoint) within a **transaction** (sequence of DML statements).
 - SAVEPOINT allows you to **rollback only a part of the transaction** instead of undoing the entire transaction.
 - SAVEPOINT is very useful when you want to manage partial rollbacks in a long transaction.
- 👉 **Key Points about SAVEPOINT:**
- A savepoint is a named point to which you can roll back during a transaction.

- You can create multiple savepoints in a single transaction.
- Rolling back to a savepoint undoes only the statements executed **after** that savepoint.
- Savepoints are released automatically when you **commit** or **rollback (without specifying a savepoint)**.
- Maximum number of savepoints depends on Oracle version (usually large, like 5 per transaction in some versions, higher in later versions).

Syntax:

SQL>SAVEPOINT savepoint_name;

Here,

Savepoint_name is a named point to which you can roll back a transaction.

Example:

Suppose we are working on an EMPLOYEE table.

-- Consider a transaction having sequence of DML commands-

```
SQL>INSERT INTO employee (emp_id, name, salary) VALUES (101, 'Amit', 50000);
SQL>SAVEPOINT sp1;
SQL>INSERT INTO employee (emp_id, name, salary) VALUES (102, 'Raj', 60000);
SQL>SAVEPOINT sp2;
SQL>INSERT INTO employee (emp_id, name, salary) VALUES (103, 'Priya', 70000);
```

Here, we have 3 insert operations with 2 savepoints in between.

Now, We fire ROLLBACK command by considering four cases-

Case 1: Rollback to sp2

SQL>ROLLBACK TO sp2;

👉 The row for **Priya (103)** will be removed, but Amit (101) and Raj (102) remain as it is.

Case 2: Rollback to sp1

SQL>ROLLBACK TO sp1;

👉 Only Amit (101) record remains as it is where as Raj (102) and Priya (103) records are removed.

Case 3: Full Rollback

SQL>ROLLBACK;

👉 All inserts (Amit, Raj, Priya) are undone.

Case 4: Commit

SQL>COMMIT;

👉 All changes saved permanently after COMMIT, and transaction becomes completed. Also, **savepoints are cleared**.

i.e here after COMMIT, **SQL>ROLLBACK TO savepoint;** command not works. That is All savepoints are erased after COMMIT

3) SET TRANSACTION command:

- The SET TRANSACTION command in Oracle is used to **initialize the properties for a transaction** i.e. it sets transaction mode and isolation level of transaction before it begins.
- This command allows you to set the mode of transaction as **READ ONLY** or **READ WRITE**
- This command also allows you to set the isolation level of transaction (how the transaction is isolated from others) as **READ COMMITTED** or **SERIALIZABLE**.

Syntax:

1) Setting mode of transaction:

SQL>SET TRANSACTION mode;

Here,

mode: represents mode of transaction that may be **READ ONLY** or **READ WRITE**

MODE	Description
READ WRITE	This is <u>default</u> transaction mode. It allows DML (INSERT, UPDATE, DELETE) operations.
READ ONLY	The transaction cannot perform any DML (INSERT, UPDATE, DELETE). Only SELECT or Read operation in transaction is carried out.

Example 1:

SQL> SET TRANSACTION READ WRITE;

The above query, sets transaction mode to READ WRITE (which is default) therefore we can do all transaction operations (insert, update, delete) very easily.

```
SQL> SET TRANSACTION READ WRITE;

Transaction set.

SQL> insert into stud values(2,'SAMIR','MADHA',9210101010);

1 row created.

SQL> select * from stud;

  ROLL_NO NAME                ADDRESS                MOBILE
-----
1 JIBRAIL                    thane                98222222
2 SAMIR                      MADHA                9210101010
```

Example 2:

SQL> SET TRANSACTION READ ONLY;

- The above query, sets transaction mode to READ ONLY therefore we cannot perform transaction operations (insert, update, delete).
- Only transaction read (SELECT) operation can be carried out successfully.

```
SQL> SET TRANSACTION READ ONLY;

Transaction set.

SQL> insert into stud values(3,'AMIR','MUMBAI',9522222222);
insert into stud values(3,'AMIR','MUMBAI',9522222222)
*
ERROR at line 1:
ORA-01456: may not perform insert/delete/update operation inside a READ ONLY
transaction

SQL> select * from stud;

  ROLL_NO NAME                ADDRESS                MOBILE
-----
1 JIBRAIL                    thane                98222222
2 SAMIR                      MADHA                9210101010
```

In above example, insert operation failed due to transaction mode is set to READ ONLY mode.

2) Setting isolation level of transaction:

SQL> SET TRANSACTION ISOLATION LEVEL levels;

Here,

levels: represents isolation level of transaction that may be **SERIALIZABLE** or **READ COMMITTED**

ISOLATION LEVEL	Description
READ COMMITTED	This is <u>default</u> isolation level. Each <u>query sees only committed data</u> as of the start of the query.
SERIALIZABLE	Ensures complete isolation. It <u>blocks other transactions.</u>

Example 1:

SQL> SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

Session-I (First RUN SQL command line)	Session-II (Second RUN SQL command line)
SQL> SET TRANSACTION ISOLATION LEVEL READ COMMITTED; <i>Transaction 1:</i>	<i>Transaction 2 (another session):</i> SQL> UPDATE employees SET salary = 7000 WHERE id = 1; SQL> COMMIT;

SQL>SELECT salary FROM employees WHERE id = 1; OUTPUT: SALARY ----- 5000	
<i>Back to Transaction 1:</i> SQL>SELECT salary FROM employees WHERE id = 1; OUTPUT: SALARY ----- 7000	♦ NOTE: Even though you're in the same transaction, the second query sees the new committed data. That's because each query gets a fresh read of committed data.

Example 2: SQL> SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

Session-I (First RUN SQL command line)	Session-II (Second RUN SQL command line)
SQL> SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; <i>Transaction 1:</i> SQL>SELECT salary FROM employees WHERE id = 1; OUTPUT: SALARY ----- 5000	<i>Transaction 2 (another session):</i> SQL>UPDATE employees SET salary = 7000 WHERE id = 1; SQL>COMMIT;
<i>Back to Transaction 1:</i> SQL>SELECT salary FROM employees WHERE id = 1; OUTPUT: SALARY ----- 5000	♦ NOTE: In Session-I, both Query 1 and Query 2 would return the same value (5000) even if another transaction committed a change.

Note:

- SET TRANSACTION must be the **first SQL statement** in a transaction. That is, we are not able to use SET TRANSACTION command after DML operations until we COMMIT transaction first.
- If auto-commit is enabled (e.g., in SQL*Plus or some tools), a transaction might be implicitly committed, and your SET TRANSACTION won't apply unless you explicitly manage transactions using BEGIN.

Explanation of all DCL (DATA Control Language) Commands:

- In Oracle, DCL (Data Control Language) commands are used **to control access the data of the database** essentially, they deal with permissions and privileges.
- DCL commands enforce database security in a multiple user database environment.
- DBA controls the access of users to the database. Only Database Administrator's or owners of the database object can add / remove privileges on a database object.

Common DCL Commands in SQL are:

DCL Command	Purpose
GRANT	Gives specific privileges to user
REVOKE	Removes privileges from a user

- In Oracle, the GRANT and REVOKE commands are Data Control Language (DCL) statements **used to manage user privileges and roles within the database.**

Let's see these commands in details-

1. GRANT Command:

- The GRANT command is used to give users **access rights (object privileges)** to database objects (tables, views, procedures, index etc.) or **system privileges** (like creating tables or users).
- Some **Object privileges** are:

Privilege	Description
SELECT	Read data from a table/view
INSERT	Insert new rows
UPDATE	Update existing rows
DELETE	Delete rows
REFERENCES	Create foreign keys using the table
INDEX	Create indexes on the table
EXECUTE	Execute a procedure/function

- Some **System privileges** are:

Privilege	Description
CREATE TABLE	Create new tables
CREATE VIEW	Create views
CREATE USER	Create new users
DROP ANY TABLE	Drop any table in any schema
ALTER ANY TABLE	Alter any table

Note:

- Only DBAs or users with sufficient privileges** can grant/revoke system privileges.
- Revoking a privilege** will **not** cascade; it removes the privilege **only from the specific user**, not others they may have granted it to.

Syntax:

SQL>GRANT privilege1, privilege2,... **ON** object **TO** user1,user2,... **[WITH GRANT OPTION];**

Here,

Part	Description
GRANT	Keyword to start granting privileges
Privilege1, privilege2,...	One or more object privileges like SELECT, INSERT, UPDATE
object	The table, view, procedure, etc., that the privilege is being granted on
user1, user2,...	One or more users or roles to whom you're granting the privileges
[WITH GRANT OPTION]	(Optional) Allows the user to grant the same privilege to others

Examples of GRANT command:

(For me: Show all below practical by opening multiple Run SQL command Line instances one for system/password and other is for newly created user)

1) Granting CONNECT privileges to USER:

SQL> GRANT CONNECT to john;

This query allows 'john' to connect with database.

2) Granting SELECT and INSERT privileges on a table:

SQL>GRANT SELECT, INSERT ON emp TO john;

This allows 'john' to view (SELECT) and add records (INSERT) to the 'emp' table of SYSTEM user.

Note: Verify SELECT and INSERT commands works or not by login via 'john' user.

E.g.

```
SQL> connect      john/john123
Connected.
SQL> select * from system.emp;
```

EMPID	EMPNAME	SAL	COMMITION
12	JIBRAIL	4000	200
2	samir	2000	
23	pratik	5000	200

Note: While accessing table of SYSTEM in other user, we have to specify 'SYSTEM' user name along with table name which is shown in above example.

3) Granting UPDDATE privileges on a table:

```
SQL>GRANT UPDATE ON emp TO john;
```

This allows 'john' to update the 'emp' table of SYSTEM user.

Note: Verify UPDATE command works or not by login via 'john' user.

E.g.

```
SQL> update system.emp set commition=300 where empid=2;

1 row updated.

SQL> select * from system.emp;
```

EMPID	EMPNAME	SAL	COMMITION
12	JIBRAIL	4000	200
2	samir	2000	300
23	pratik	5000	200

Note: While accessing table of SYSTEM in other user, we have to specify 'SYSTEM' user name along with table name which is shown in above example.

4) Granting system privileges:

```
SQL>GRANT CREATE TABLE TO hr;
```

Now, the user 'hr' can create tables.

5) Granting a role:

```
SQL>GRANT dba TO scott;
```

Gives all privileges in the DBA role to user scott.

6) Using WITH GRANT OPTION:

```
SQL>GRANT SELECT ON employees TO sam WITH GRANT OPTION;
```

Now, 'sam' can grant the SELECT privilege on employees to other users.

7) Granting SELECT (read) all tables to user 'john':

```
SQL>GRANT SELECT any table to john;
```

Here, Grant SELECT privileges of all tables to user 'john'

1) Creating new database user:

Syntax: SQL> CREATE USER username IDENTIFIED BY password;

The above query, creates new database user with username and password.

Example-

```
SQL> CREATE USER john IDENTIFIED BY john123;
```

The above query, creates new database user 'john' having password 'john123'

After, creation of user we try to login with 'john' user as follow. But it would generate error like-

```
SQL> connect john/john123
ERROR:
ORA-01045: user JOHN lacks CREATE SESSION privilege; logon denied
```

Imp Note:

When new user is created, it does not have any rights related with database.

Here, only new database user is created. This created user is not capable to connect the database and not performs any operations on database. Therefore, we have to assign several predefined roles for created user which are discussed below-

Common Predefined User Roles with typical privileges (Freedom):

User Role Name	Description	Typical Privileges	Use Case / Notes
CONNECT	Basic role that allows a user to connect to the database.	CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE SEQUENCE, etc.	For users needing basic access to begin working in their own schema.
RESOURCE	This role allowing creation of additional schema objects like creating table, creating view etc.	Includes CREATE PROCEDURE, CREATE TRIGGER, CREATE TYPE, CREATE INDEX, etc.	For developers managing application logic in their own schema.
DBA	Full administrative privileges for the entire database, with the ability to grant privileges to others.	All system privileges with ADMIN OPTION	Should be granted only to trusted DBAs due to full control over the database.
EXP_FULL_DATABASE	Role designed for performing full database exports.	Privileges needed to export any schema's data and metadata.	Used in backup, migration, or data archival tasks.
IMP_FULL_DATABASE	Role designed for performing full database imports.	Privileges needed to import data into any schema , overriding normal access controls.	Used for restoring data or importing large database dumps.

Granting or Assigning CONNECT role to created john user-

```
SQL> GRANT CONNECT TO john;

Grant succeeded.
```

Now, user 'john' able to connect to oracle database as-

```
SQL> connect john/john123
Connected.
```

Here, we create new table under logged user 'john' as-

```
SQL> create table account_info(
  2 acc_no number,
  3 acc_type varchar2(10),
  4 acc_holder varchar2(15),
  5 balance number);
create table account_info(
*
ERROR at line 1:
ORA-01031: insufficient privileges
```

The above query generates error 'insufficient privileges'. This is due to 'john' user does not have privileges to create new table. We have to assign 'CREATE TABLE' privileges to john user as-

```
SQL> grant CREATE TABLE TO john;

Grant succeeded.
```

Now, we try to create new table as-

```
SQL> create table account_info(
  2 acc_no number,
  3 acc_type varchar2(10),
  4 acc_holder varchar2(15),
  5 balance number);
create table account_info(
*
ERROR at line 1:
ORA-01950: no privileges on tablespace 'SYSTEM'
```

Again, it generates error 'no privileges on tablespace 'SYSTEM''. This is due to we have not assign tablespace along with memory size (quota) for 'john' user.

We can assign tablespace along with memory size for 'john' user by altering it as-

```
SQL> ALTER USER john QUOTA UNLIMITED ON SYSTEM;

User altered.
```

Now, again we try to create new table as-

```
SQL> create table account_info(
  2 acc_no number,
  3 acc_type varchar2(10),
  4 acc_holder varchar2(15),
  5 balance number);

Table created.
```

Here, new table creates successfully under 'john' user.

Note: If we have assigned DBA role to created user then there is no need to assign several privileges to created user. Because, DBA role has Full administrative privileges for the entire database, with the ability to grant privileges to others also.

Consider following example, we create new user 'tom' and assigning DBA role as –


```
SQL> create user tom identified by tom123;

User created.

SQL> grant DBA to tom;

Grant succeeded.

SQL> connect tom/tom123;
Connected.
SQL> create table stud_mast(
  2  roll  number,
  3  name  varchar2(15),
  4  class varchar2(10),
  5  per   number(5,2));

Table created.
```

Here, tom is DBA user that have all rights same as SYSTEM. No need to specify extra privileges to tom.

2. REVOKE Command:

- In Oracle, the REVOKE command is used to **remove previously granted privileges or roles** from a user from database.
- This helps in tightening security or controlling access when a user should no longer have certain permissions.

- **Syntax:**

SQL>REVOKE privilege1, privilege2,... **ON** object **FROM** user1,user2,...;

- **Here,**

Part	Description
REVOKE	Is Keyword to start revoking assigned privileges
Privilege1, privilege2,...	One or more object privileges like SELECT, INSERT, UPDATE
object	The table, view, procedure, etc., that the privilege is being granted on
user1, user2,...	One or more users or roles from whom you're revoking the privileges

1. Revoke Object Privilege:

Suppose SYSTEM user has granted SELECT and INSERT on the EMP table to user SCOTT.

To revoke these:

SQL> REVOKE SELECT, INSERT **ON** EMP **FROM** SCOTT;

Above query revoke SELECT and INSERT privileges On table EMP from user SCOTT.

2. Revoke System Privilege:

SQL>REVOKE CREATE TABLE **FROM** SCOTT;

Above query revoke CREATE TABLE privileges from user SCOTT.

3. Suppose SYSTEM user has granted SELECT on all tables to user john, to revoke these:

SQL> REVOKE SELECT any table **FROM** john;

Note:

- You can only revoke privileges that you granted, unless you're a DBA.
- When a privilege is revoked, the user loses that access immediately.

SPOOL in Oracle-

In Oracle, SPOOL is a SQL*Plus (or SQLcl) command used to write the output of queries or scripts to a file. It's commonly used to export results to .txt, .csv, or .log files.

Basic Syntax to Use SPOOL:

SQL>SPOOL file_path\filename.extension

E.g. **SQL>SPOOL** d:\3301\practdbms.txt

The above query will save the all outputs of statements into file **d:\3301\practdbms.txt**.

Note:

- 1) If file is not present at location, it will create and output will save in it.
- 2) If file is already present at location, it will clear all previous data and new output will save in it.

At last, make spool off as- (This will saves all data in spool file)

SQL>SPOOL OFF

SQL Operators used in Oracle-

- **Operator-** Operator is symbol that operates onto operand and performs specific task.
- Every operator has its own task which is predefined.
- Oracle provides a wide range of operators used in SQL statements to perform operations on data. These operators are grouped into several categories, each serving different purposes which is shown as follow-

Category	Description
Arithmetic Operators	Perform mathematical operations
Comparison (Relational) Operators	Compare values
Logical Operators	Performs logical operations
Set Operators	Combines result sets of multiple queries

Let's see all these operators in details-

✓ 1. Arithmetic Operators

These operators are used to perform mathematical calculations shown in following table-

Operator	Description	Example
+	Addition	SQL>SELECT 10 + 5 FROM DUAL;
-	Subtraction	SQL>SELECT 10 - 5 FROM DUAL;
*	Multiplication	SQL>SELECT 10 * 5 FROM DUAL;
/	Division	SQL>SELECT 10 / 5 FROM DUAL;

✓ 2. Comparison (Relational) Operators:

These operators are used to compare two values. Returns TRUE, FALSE, or NULL.

All comparison operators are explained in following table-

Operator	Description	Example
=	Equal to	SQL>SELECT * FROM EMP WHERE SAL = 5000;
!= or <>	Not equal to	SQL>SELECT * FROM EMP WHERE SAL <> 10;
>	Greater than	SQL>SELECT * FROM EMP WHERE SAL > 3000;
<	Less than	SQL>SELECT * FROM EMP WHERE SAL < 3000;
>=	Greater than or equal	SQL>SELECT * FROM EMP WHERE SAL >= 3000;
<=	Less than or equal	SQL>SELECT * FROM EMP WHERE SAL <= 3000;
BETWEEN	Within a range	SQL>SELECT * FROM EMP WHERE SAL BETWEEN 2000 AND 5000;
LIKE	Pattern matching	SQL>SELECT * FROM EMP WHERE ENAME LIKE 'S%';
IN	Match in a list (Either Or) (Specific record selection from list)	SQL>SELECT * FROM EMP WHERE ENO IN (10, 20);
IS NULL	Check for NULL value	SQL>SELECT * FROM EMP WHERE COMMISSION IS NULL;

Homework:

- 1) Create table EMP having fields eno,ename,sal,eadd,commission with appropriate data type and size.
- 2) Insert any 15 records in created table EMP.
- 3) Run queries shown in above table and write output.

❖ Wildcard characters percentage (%) and underscore (_)

- In Oracle SQL, the wildcard characters % and _ are used with the LIKE operator for pattern matching.

Wildcard character Percentage (%) :

The % wildcard represents zero or more characters in the string.

Example: LIKE 'S%'

This matches any string that starts with the letter 'S' and can be followed by any number of characters (including none).

- Matches: 'Smith', 'Scott', 'S', 'Superman'
- Does not match: 'Jones', 'Allen'

Wildcard character Underscore (_) :

The (_) wildcard represents **exactly one character** in the string.

Example: LIKE 'S_'

This matches any string that starts with 'S' and is followed by exactly one character.

- Matches: 'Sm', 'Sz', 'St'
- Does not match: 'S', 'Scott'.

We can combine both wildcards in a single query for more complex patterns shown in following examples

Example 1: LIKE 'S_%_a'

This matches strings that:

- Start with 'S'
- Have zero or more characters in the middle (because of the %)
- End with 'a'
- Matches: 'Samantha', 'Socotyza'
- Does not match: 'Smith', 'S', 'Sarah'

Example 2: LIKE 'S__%n'

This matches strings that:

- 1) Must start with the letter S.
- 2) Any two characters after S
- 3) '%' → Matches **zero or more characters**.
- 4) 'n' → The string must **end with the letter 'n'**, after any number of characters matched by %

Examples of Matching Strings pattern:

- Samson, StuwN, S14winn, Sxxn, S__n, Spoon, Sopan, Soon

Examples that Do Not Match String pattern:

- San, Steve (doesn't end with n), stiven (doesn't start with capital S), Soap

✓ 3. Logical Operators

Logical operators are used to combine multiple conditions.

All Logical operators are explained in following table-

Operator	Description	Example
AND	All conditions must be true	SELECT * FROM EMP WHERE DEPTNO = 10 AND SAL > 3000
OR	At least one condition must be true	SELECT * FROM EMP WHERE DEPTNO = 10 OR DEPTNO = 20
NOT	Negates the condition	SELECT * FROM EMP WHERE NOT (SAL > 3000) Hint: Display employees whose SAL not greater than 3000 (it selects 3000 too because 3000 not greater than 3000)

✓ 4. Set Operators:

Used to combine the result of two SELECT statements.

All set operators are explained in following table-

Operator	Description	Example
UNION	Combines results, removes duplicates	SELECT DEPTNO FROM EMP UNION SELECT DEPTNO FROM DEPT;
UNION ALL	Combines all results (with duplicates)	SELECT DEPTNO FROM EMP UNION ALL SELECT DEPTNO FROM DEPT;
INTERSECT	Shows common records	SELECT DEPTNO FROM EMP INTERSECT SELECT DEPTNO FROM DEPT;
MINUS	Shows records in first query after getting minus from second query.	SELECT DEPTNO FROM EMP MINUS SELECT DEPTNO FROM DEPT;

Built in functions in ORACLE:

- We know that function is block of statement that has specific task. Programming task becomes easy due

to function.

- Oracle SQL has a wide range of **built-in functions** that fall into several categories which are listed below-
 - 1) Arithmetic Functions
 - 2) String (Character) Functions
 - 3) Date and Time Functions
 - 4) Conversion Functions
 - 5) Aggregate Functions
 - 6) General Functions

Let's see these Functions in details-

1) Arithmetic Functions-

- In Oracle, **arithmetic functions** are used to perform mathematical operations on numeric data. These are typically **built-in SQL functions** that return numeric results.
- Commonly used arithmetic functions are shown in following table-

Function	Description	Example
ABS()	Absolute value	ABS(-15)=15
CEIL()	Returns <u>upper integer</u> value	CEIL(14.6)=15
FLOOR()	Returns <u>lower integer</u> value	FLOOR(8.4)=8
MOD()	Remainder of division	MOD(5,3)=2
POWER()	Exponentiation	POWER(4,3)=64
ROUND()	Round to decimal places	ROUND(45.154736,3)=45.155
TRUNC()	Truncate decimals	TRUNC(45.154736,3)=45.154
SQRT()	Square root	SQRT(64)=8
SIGN()	Sign of number (-1, 0, 1)	SIGN(-15)=-1, SIGN(17)=1, SIGN(0)= 0
EXP()	Exponential of number	EXP(1) = 2.7182818
LN()	Natural logarithm	LN(10)= 2.302585...
LOG()	Logarithm with custom base	LOG(10, 100)=2
GREATEST()	Largest of list	GREATEST(14,28,35,16)=35
LEAST()	Smallest of list	LEAST(30,58,4,36)=4

- Let's see these functions in detail with **examples**.

1. ABS() – Absolute Value

Description: Returns the **absolute (positive) value of a number**.

Syntax: ABS(n)

Example: SQL>SELECT ABS(-45) AS Result FROM dual;
-- Result: 45

2. CEIL() – Ceiling

Description: Returns upper integer value of fractional number. (i.e. returns **integer value greater than or equal to the number**.)

Syntax: CEIL(n)

Example: SQL>SELECT CEIL(12.3) AS Result FROM dual;
-- Result: 13

3. FLOOR() – Floor

Description: Returns lower integer value of fractional number. (i.e. returns the **integer value less than or equal to the number**.)

Syntax: FLOOR(n)

Example: SQL>SELECT FLOOR(12.9) AS Result FROM dual;
-- Result: 12

4. MOD() – Modulus

Description: Returns the remainder of a division operation.

Syntax: MOD(n1, n2)

Example: SQL>SELECT MOD(10, 3) AS Result FROM dual;
-- Result: 1

5. POWER() – Exponentiation

Description: Returns the value of one number raised to the power of another. (i.e. it calculates base raise to power)

Syntax: POWER(n, m)

Example: SQL>SELECT POWER(2, 3) AS Result FROM dual;
-- Result: 8

6. ROUND() – Round Number

Description: Rounds a number to a specified number of decimal places.

Syntax: ROUND(n, decimal_places)

Example: SQL>SELECT ROUND(123.456, 2) AS Result FROM dual;
-- Result: 123.46

7. TRUNC() – Truncate Number

Description: Truncates a number to a specified number of decimal places without rounding.

Syntax: TRUNC(n, decimal_places)

Example: SQL>SELECT TRUNC(123.456, 2) AS Result FROM dual;
-- Result: 123.45

8. SQRT() – Square Root

Description: Returns the square root of a number.

Syntax: SQRT(n)

Example: SQL>SELECT SQRT(16) AS Result FROM dual;
-- Result: 4

9. SIGN() – Sign Indicator

Description: Returns -1, 0, or 1 depending on whether the number is negative, zero, or positive.

Syntax: SIGN(n)

Example: SQL>SELECT SIGN(-10) AS Result FROM dual; Result -1
SQL>SELECT SIGN(0) AS Result FROM dual; Result 0
SQL>SELECT SIGN(10) AS Result FROM dual; Result 1

10. EXP() – Exponential

Description: Returns e (Euler's number) raised to the power of the input number.

Syntax: EXP(n)

Example: SQL>SELECT EXP(1) AS Result FROM dual;
-- Result: 2.7182818

11. LN() – Natural Logarithm

Description: Returns the natural logarithm (base e) of a number.

Syntax: LN(n)

Example: SQL>SELECT LN(10) AS Result FROM dual;
-- Result: 2.302585...

12. LOG() – Logarithm with Custom Base

Description: Returns the logarithm of a number with a specific base.

Syntax: LOG(base, number)

Example: SQL>SELECT LOG(10, 100) AS Result FROM dual;
-- Result: 2

13. GREATEST() and LEAST()

Description:

- GREATEST() returns the largest value among two or more inputs.
- LEAST() returns the smallest value among two or more inputs.

Syntax:

GREATEST(n1, n2, ..., nN)

LEAST(n1, n2, ..., nN)

Example: SQL>SELECT GREATEST(10, 20, 5) AS max_val FROM DUAL,
-- max_val = 20
SQL> SELECT LEAST(10, 20, 5) AS min_val FROM DUAL; -- min_val = 5

2) String (Character) Functions:

- In Oracle SQL, **String (Character) Functions** are used to manipulate string data (VARCHAR, VARCHAR2, CHAR, etc.).
- These functions help to format, modify, or retrieve specific portions of string values.
- Here's a list of commonly used **String Functions in Oracle SQL**, along with their **descriptions** and **examples**:

Function	Description	Example SQL	Output
UPPER(str)	Converts all letters to uppercase.	SQL>SELECT UPPER('oracle') FROM dual;	ORACLE
LOWER(str)	Converts all letters to lowercase.	SQL>SELECT LOWER('OrAcLe') FROM dual;	oracle
INITCAP(str)	Capitalizes the first letter of each word.	SQL>SELECT INITCAP('oracle sql') FROM dual;	Oracle Sql
LENGTH(str)	Returns the number of characters.(including space)	SQL>SELECT LENGTH('Oracle') FROM dual;	6
SUBSTR(str, start, len)	Returns substring starting from a position.	SQL>SELECT SUBSTR('OracleSQL', 1, 6) FROM dual;	Oracle
INSTR(str, substr)	Finds the position of a substring.	SQL>SELECT INSTR('OracleSQL', 'S') FROM dual;	7
LPAD(str, len, padchar)	Left-pads string to given length.	SQL>SELECT LPAD('123', 5, '*') FROM dual;	**123
RPAD(str, len, padchar)	Right-pads string to given length.	SQL>SELECT RPAD('123', 5, '*') FROM dual;	123**
TRIM(char FROM str)	Removes leading/trailing (eigher first or last character).	SQL>SELECT TRIM('O' FROM 'Oracle') FROM dual;	racle
REPLACE(str, search, replace)	Replaces substrings.	SQL>SELECT REPLACE('Oracle DB', 'DB', 'Cloud') FROM dual;	Oracle Cloud
CONCAT(str1, str2)	Concatenates two strings.	SQL>SELECT CONCAT('Oracle', 'SQL') FROM dual;	OracleSQL
TRANSLATE(str, mat_char, map_char)	Replaces each character by mapping.	SQL>SELECT TRANSLATE('12345', '2', 'X') FROM dual;	1X345
ASCII(char)	Returns ASCII value of the character.	SQL>SELECT ASCII('A') FROM dual;	65
CHR(code)	Returns character for ASCII code.	SQL>SELECT CHR(65) FROM dual;	A

Let's see all these functions in details-

1. UPPER(string)

Converts all characters in a string to uppercase.

Example- SQL>SELECT UPPER('oracle') AS upper_example FROM dual;

-- Output: ORACLE

2. LOWER(string)

Converts all characters in a string to lowercase.

Example- SQL>SELECT LOWER('OrAcLe') AS lower_example FROM dual;

-- Output: oracle

3. INITCAP(string)

Converts the first character of each word to uppercase and the rest to lowercase.

Example- SQL>SELECT INITCAP('welcome to oracle') AS initcap_example FROM dual;

-- Output: Welcome To Oracle

4. LENGTH(string)

Returns the number of characters in the string.

Example- **SQL>SELECT** LENGTH('Oracle') AS length_example FROM dual;
-- Output: 6

5. SUBSTR(string, start_position, length)

Returns a part (substring) of a string starting at a given position.

Example- **SQL>SELECT** SUBSTR('Oracle Database', 1, 6) AS substr_example FROM dual;
-- Output: Oracle

6. INSTR(string, substring)

Returns the position of the first occurrence of a substring in a string.

If substring is not found in main string then it returns 0.

Example- **SQL>SELECT** INSTR('Oracle Database', 'D') AS instr_example FROM dual;
-- Output: 8

7. LPAD(string, length, pad_char)

Left-pads a string with a character to reach a certain length.

Example- **SQL>SELECT** LPAD('123', 5, '0') AS lpad_example FROM dual;
-- Output: 00123

8. RPAD(string, length, pad_char)

Right-pads a string with a character to reach a certain length.

Example- **SQL>SELECT** RPAD('123', 5, '0') AS rpad_example FROM dual;
-- Output: 12300

9. TRIM([LEADING | TRAILING | BOTH] trim_char FROM string)

Removes specified characters (or spaces by default) from the beginning, end, or both ends of a string.

Example- **SQL>SELECT** TRIM(' x' FROM 'xOraclex') AS trim_example FROM dual;
-- Output: Oracle

Example- **SQL>SELECT** TRIM('O' FROM 'Oracle') AS trim_char_example FROM dual;
-- Output: racle

10. REPLACE(string, search_string, replacement_string)

Replaces all occurrences of a substring with another substring.

Example- **SQL>SELECT** REPLACE('Oracle Dbase', 'Dbase', 'Cloud') AS replace_example FROM dual;
-- Output: Oracle Cloud

11. CONCAT(string1, string2)

Concatenates two strings. (Only two arguments allowed)

Example- **SQL>SELECT** CONCAT('Oracle', 'DB') AS concat_example FROM dual;
-- Output: OracleDB

12. TRANSLATE(string, from_chars, to_chars)

Replaces each character in from_chars with the corresponding character in to_chars.

Example- **SQL>SELECT** TRANSLATE('12345', '123', 'abc') AS translate_example FROM dual;
-- Output: abc45

13. ASCII(char)

Returns the ASCII code of the first character.

Example- **SQL>SELECT** ASCII('A') AS ascii_example FROM dual;
-- Output: 65

14. CHR(ascii_code)

Returns the character corresponding to the ASCII code.

Example- **SQL>SELECT** CHR(65) AS chr_example FROM dual;
-- Output: A

3) Date and Time Functions:

- Oracle provides a rich set of **Date and Time functions** to handle **date, time, timestamp, intervals**, and **time zone operations**.
- Following table shows different date and time functions with description and example-

Function Name	Description	Example
SYSDATE	Returns current date and time of DB server	SELECT SYSDATE FROM DUAL;
SYSTIMESTAMP	Returns current timestamp with time zone and fractional seconds of DB server	SELECT SYSTIMESTAMP FROM DUAL;
CURRENT_DATE	Returns current date in session time zone.	SELECT CURRENT_DATE FROM DUAL;
CURRENT_TIMESTAMP	Returns current timestamp in session time zone.	SELECT CURRENT_TIMESTAMP FROM DUAL;
LOCALTIMESTAMP	Returns current timestamp (no time zone info).	SELECT LOCALTIMESTAMP FROM DUAL;
DBTIMEZONE	Returns server database time zone .	SELECT DBTIMEZONE FROM DUAL;
SESSIONTIMEZONE	Returns session time zone.	SELECT SESSIONTIMEZONE FROM DUAL;
TO_DATE	Converts string to date.	SELECT TO_DATE ('2025-09-04', 'YYYY-MM-DD') FROM DUAL;
TO_CHAR	Converts date/timestamp to string with format.	SELECT TO_CHAR (SYSDATE, 'DD-MON-YYYY HH24:MI:SS') FROM DUAL;
TO_TIMESTAMP	Converts string to timestamp.	SELECT TO_TIMESTAMP ('2025-09-04 10:00:00', 'YYYY-MM-DD HH24:MI:SS') FROM DUAL;
ADD_MONTHS	Adds or subtracts months to/from a date. (if month val is +ve then months add, if month val is -ve then months subtracts)	SELECT ADD_MONTHS (SYSDATE, 2) FROM DUAL;
MONTHS_BETWEEN	Returns number of months between two dates.	SELECT MONTHS_BETWEEN (SYSDATE, TO_DATE ('2025-01-01', 'YYYY-MM-DD')) FROM DUAL;
LAST_DAY	Returns last day of the month for given date. (e.g. 30-Sep-2025)	SELECT LAST_DAY (SYSDATE) FROM DUAL;
NEXT_DAY	Returns next occurrence of given weekday.	SELECT NEXT_DAY (SYSDATE, 'FRIDAY') FROM DUAL;
ROUND(date)	Rounds date to nearest unit (day, month, etc.).	SELECT ROUND (SYSDATE, 'MONTH') FROM DUAL;
EXTRACT	Extracts part (YEAR, MONTH, DAY) from date or timestamp.	SELECT EXTRACT (YEAR FROM SYSDATE) FROM DUAL;
NEW_TIME	Converts date from one time zone to another (legacy).	SELECT NEW_TIME (SYSDATE, 'PST', 'GMT') FROM DUAL;
FROM_TZ	Assigns time zone to a timestamp.	SELECT FROM_TZ (TIMESTAMP '2025-09-04 10:00:00', ' UTC ') FROM DUAL;
INTERVAL (literal)	Adds specific interval duration to timestamp.	SELECT SYSTIMESTAMP + INTERVAL '2' DAY FROM DUAL;
DATE arithmetic	Subtracting dates returns difference in days.	SELECT SYSDATE - TO_DATE ('2025-09-01', 'YYYY-MM-DD') AS days_diff FROM DUAL;

Common Date Format Elements in Oracle:

Format	Meaning	Example
YYYY	4-digit year	1985
YY	2-digit year	85
MM	2-digit month	06
MON	3-letter month	JUN
MONTH	Full month name	JUNE
DD	Day of month	20
DY	3-letter day	THU
DAY	Full day name	Thursday

HH24	24-hour format	17
HH	12-hour format	5 AM
MI	Minute	45
SS	Second	25

We explain all above-mentioned date and functions as follow-

◆ 1. SYSDATE

Returns the current date and time of the **database server** (no fractional seconds).

SQL>SELECT SYSDATE FROM DUAL;

Output example: 04-SEP-25 10:24:00

◆ 2. SYSTIMESTAMP

Returns the current timestamp with **time zone and fractional seconds**.

SQL>SELECT SYSTIMESTAMP FROM DUAL;

Output example: 04-SEP-25 10.24.00.123456 AM +05:30

◆ 3. CURRENT_DATE

Returns the current date and time **in the session time zone**.

SQL>SELECT CURRENT_DATE FROM DUAL;

◆ 4. CURRENT_TIMESTAMP

SQL>**Returns** current timestamp **in the session time zone**.

SELECT CURRENT_TIMESTAMP FROM DUAL;

◆ 5. LOCALTIMESTAMP

Returns the current timestamp without time zone.

SQL>SELECT LOCALTIMESTAMP FROM DUAL;

◆ 6. DBTIMEZONE

Returns the database server time zone as a string.

SQL>SELECT DBTIMEZONE FROM DUAL;

◆ 7. SESSIONTIMEZONE

Returns the current session's time zone.

SQL>SELECT SESSIONTIMEZONE FROM DUAL;

◆ 8. EXTRACT

Extracts and returns a **specific part** of a date (like year, month, day).

SQL>SELECT EXTRACT(YEAR FROM SYSDATE) AS YEAR FROM DUAL;

SQL>SELECT EXTRACT(MONTH FROM SYSTIMESTAMP) AS MONTH FROM DUAL;

◆ 9. TO_DATE

Converts a string to a **DATE** type.

SQL>SELECT TO_DATE('2025-09-04', 'YYYY-MM-DD') FROM DUAL;

◆ 10. TO_CHAR

Converts a date to a **string** with the specified format.

SQL>SELECT TO_CHAR(SYSDATE, 'DD-MON-YYYY HH24:MI:SS') FROM DUAL;

◆ 11. TO_TIMESTAMP

Converts a string to a **TIMESTAMP**.

SQL>SELECT TO_TIMESTAMP('2025-09-04 10:30:00', 'YYYY-MM-DD HH24:MI:SS') FROM DUAL;

◆ 12. ADD_MONTHS

Adds or **subtracts** months to a date.

SQL>SELECT ADD_MONTHS(SYSDATE, 2) FROM DUAL;

SQL>SELECT ADD_MONTHS(SYSDATE, -1) FROM DUAL;

◆ 13. MONTHS_BETWEEN

Returns the number of months between two dates.

SQL>SELECT MONTHS_BETWEEN (SYSDATE, TO_DATE ('2025-01-01', 'YYYY-MM-DD')) FROM DUAL;

◆ 14. NEXT_DAY

Finds the **next date** that falls on a specific **weekday**.

SQL>SELECT NEXT_DAY (SYSDATE, 'FRIDAY') FROM DUAL;

◆ 15. LAST_DAY

Returns the **last day** of the month for a date. (**dd-MON-YY** format)

SQL>SELECT LAST_DAY(SYSDATE) FROM DUAL;

◆ 16. ROUND(date)

Rounds a date to the nearest unit (day, month, year, etc.)

SQL>SELECT ROUND (SYSDATE, 'MONTH') FROM DUAL;

◆ 17. NEW_TIME

Converts time from one time zone to another (legacy; limited zones).

SQL>SELECT NEW_TIME (SYSDATE, 'EST', 'GMT') FROM DUAL;

◆ 18. INTERVAL

You can use **interval literals** to add time.

SQL>SELECT SYSTIMESTAMP + INTERVAL '2' DAY FROM DUAL;

SQL>SELECT SYSTIMESTAMP + INTERVAL '10' MINUTE FROM DUAL;

◆ 19. FROM_TZ

Converts a **TIMESTAMP** to a **TIMESTAMP WITH TIME ZONE**.

SELECT FROM_TZ(TIMESTAMP '2025-09-04 10:00:00', 'UTC') FROM DUAL;

◆ 20. CURRENT_DATE - DATE

You can do **date arithmetic**: subtracting dates gives difference in **days**.

SQL>SELECT SYSDATE - TO_DATE('2025-09-01', 'YYYY-MM-DD') AS days_diff FROM DUAL;

4) Conversion Functions: (Type conversion functions)

- Oracle provides several **conversion functions** to convert data from one type to another, such as converting strings to numbers, numbers to strings, or dates to strings, and vice versa.
- These are essential when handling data with mismatched types in queries or for formatting outputs
- Following table shows **most commonly used Oracle conversion functions**, with examples.

Function	Converts From → To	Common Usage Example
TO_CHAR	Number/Date → String	TO_CHAR (SYSDATE, 'DD-MON-YYYY')
TO_NUMBER	String → Number	TO_NUMBER ('12345.67')
TO_DATE	String → Date	TO_DATE ('06-09-2025', 'DD-MM-YYYY')
TO_TIMESTAMP	String → Timestamp	TO_TIMESTAMP ('2025-09-06 14:30:45', ...)
CAST	Any supported types	CAST ('123' AS NUMBER)

Let's see these functions in details-

1. TO_CHAR:

Used to convert **numbers or dates to a string**.

Syntax: TO_CHAR (expr, [format])

Examples:

- 1) -- Convert number to string with formatting

SQL>SELECT TO_CHAR (12345.67, '99999.99') AS Result FROM dual;

- 2) -- Convert date to string

SQL>SELECT TO_CHAR(SYSDATE, 'DD-MONTH-YYYY') AS Result FROM dual;

2. TO_NUMBER:

Used to convert a **string to a number**.

Syntax: TO_NUMBER (expr, [format])

Examples:

- 1) -- Convert string to number

SQL>SELECT TO_NUMBER ('12345.67') AS num FROM dual;

- 2) -- Convert formatted string to number

SQL>SELECT TO_NUMBER ('12,345.67', '99,999.99') AS Result FROM dual;

3. TO_DATE:

Used to convert a **string into a date** using a specific format.

Syntax: TO_DATE(expr, format)

Examples:

1) -- Convert string to date
SQL>SELECT TO_DATE ('06-09-2025', 'DD-MM-YYYY') AS Result FROM dual;
 2) -- Convert another format
SQL> SELECT TO_DATE ('2025/09/06 14:30', 'YYYY/MM/DD HH24:MI') AS Result FROM dual;

4. TO_TIMESTAMP:

Converts a **string to a timestamp**, which includes fractional seconds.

Syntax: TO_TIMESTAMP(expr, format)

Example:

SQL>SELECT TO_TIMESTAMP ('2025-09-06 14:30:45.123', 'YYYY-MM-DD HH24:MI:SS.FF3') AS ts FROM dual;

5. CAST :

Used to convert **between different data types**, similar to **TO_***, but it is more flexible and ANSI SQL compliant.

Syntax: CAST(expression AS target_data_type)

Examples:

1) -- Cast string to number

SQL>SELECT CAST('123.45' AS NUMBER(6,2)) AS cast_num FROM dual;

2) -- Cast string to date

SQL>SELECT CAST('2025-09-06' AS DATE) AS cast_date FROM dual;

3) -- Cast number to character

SQL>SELECT CAST(12345 AS VARCHAR2(10)) AS cast_str FROM dual;

5) Aggregate Functions:

- Oracle **aggregate functions** perform a calculation on a set of values and return a single value.
- These functions are commonly used with the GROUP BY clause but can also be used without it to compute summary values across entire tables or result sets.
- Following table shows **most commonly used Oracle aggregate functions**, with examples.

Function	Description	Example
SUM(expr)	Total sum	SUM(salary)
AVG(expr)	Average value	AVG(salary)
MIN(expr)	Minimum value	MIN(hire_date)
MAX(expr)	Maximum value	MAX(hire_date)
COUNT(*)	Total number of rows (record)	COUNT(*)
COUNT(expr)	Count of non-null values	COUNT(salary)
COUNT(DISTINCT field)	Count of unique values	COUNT(DISTINCT department_id)

Let's see these functions in details-

Consider we have employees table-

Employees Table				
eid	ename	salary	deptId	deptName
emp101	ROHIT	5000	1	IT
emp102	KIRAN	3000	2	HR
emp103	RAJ	2000	3	Sales
emp104	UMESH	2200	2	HR
emp105	AMIR	2800	2	HR
emp106	AMOL	1570	3	Sales
emp107	SANJAY	6000	1	IT

1. SUM(): Finds the Total Sum of all Values

SQL>SELECT SUM(salary) AS total_salary FROM employees;

 Returns the total of all salaries in the employees table.

2. AVG(): Finds the Average Value of given field.

SQL>SELECT AVG(salary) AS average_salary FROM employees;
3. COUNT() – It counts the Number of Rows i.e. records as well as non-null values in table

-- Count all rows

SQL>SELECT COUNT(*) AS total_employees FROM employees;

-- Count non-null values only

SQL>SELECT COUNT(deptName) AS Depts FROM employees;

4. MIN(): Finds the Minimum Value of given field.

SQL>SELECT MIN(salary) AS MinSalary from DUAL;

5. MAX(): Finds the Maximum Value of given field.

SQL>SELECT MAX(salary) AS MaxSalary from DUAL;

6. COUNT(DISTINCT): It Counts Unique Values from given field.

SQL>SELECT COUNT(DISTINCT deptName) AS unique_dept FROM employees;

6) General Functions:

- Oracle provides a variety of **general functions** which are used to manipulate data, handle nulls, perform type conversions, and more.
- These functions are commonly used in SQL queries to customize output or perform logic inside queries.
- The commonly used general functions in oracle are shown below-

Function	Purpose	Example
NVL	Replace column NULL by specified value	NVL(col, 0)
NVL2	If column value is NOT NULL then return true otherwise false	NVL2(col, 'true', 'false')
NULLIF	Returns NULL if column value is equal to given value.	NULLIF(col, 10)
COALESCE	Returns the first non-NULL expression in the list.	COALESCE(null, null, 'A')
DECODE	Acts like a simple CASE statement.	DECODE(col, 1, 'One', 2, 'Two', 'Other')
SYSDATE	Returns Current date	SYSDATE
USER,UID	Returns Current username and its userid	USER,UID

Let's see these functions in details-

1. NVL():

Purpose: Replace NULL with a default value.

Syntax: NVL(col, replacement_value)

Example: **SQL>SELECT** eid, NVL(ename, 0) AS Result FROM employees;

This replaces NULL ename with 0.

2. NVL2():

Purpose: If column value is **NOT** NULL then return **true** otherwise **false**.

Example: **SQL>SELECT** eid, NVL2(ename, 'Yes', 'No') AS status FROM employees;

3. NULLIF():

Purpose: Returns NULL if column value is equal to given value.

Example: **SQL>SELECT** NULLIF(empid, 101) AS Result FROM dual;

4. COALESCE():

Purpose: Returns the first non-NULL expression in the list.

Example: **SQL>SELECT** COALESCE (NULL, NULL, 'Hello', 'World') AS Result FROM dual;

5. DECODE():

Purpose: Acts like a simple CASE statement.

Syntax: DECODE(expr, search1, result1, search2, result2, ..., default);

Example:

SQL>SELECT eid, DECODE(ename, 'raj', 'YES', 'sam', 'YES', 'No') AS Result FROM employees;

6. SYSDATE:

Purpose: Get current date.

Example: **SQL>SELECT** SYSDATE AS current_date FROM dual;

7. USER, UID:

Purpose: Return username and userid of the current user respectively.

Example: **SQL>SELECT** USER, UID FROM dual;

Clauses in Oracle-

- In Oracle SQL, **clauses** are components or parts of SQL statements that perform specific functions or task like filtering, grouping, sorting, or joining data.
- Understanding these clauses helps you to write powerful and efficient SQL queries.
- Let's see different clauses used in oracle with example as follow-

1) ORDER BY Clause:

- The ORDER BY clause in Oracle SQL is used to **sort the result set** of a query by one or more columns, in **ascending (ASC)** or **descending (DESC)** order.
- By default, order of sorting is Ascending order (ASC).
- **Note:**
 - ORDER BY is always applied **after** all other clauses (like WHERE, GROUP BY, etc.).
 - If ORDER BY is not specified, Oracle does **not guarantee any specific order** of results.

Syntax-

```
SQL>SELECT col1, col2, ... FROM table_name ORDER BY col1 SortOrd, col2 SortOrd, ...;
```

Here,

SortOrd may be **DESC** (To sort data in Descending order) or **ASC** (Ascending order)

Example: Suppose we have 'employees' table as-

emp_id	first_name	salary
101	John	5000
102	Alice	7000
103	Bob	6000

✅ Example 1: Sort by Salary (Ascending)

```
SQL>SELECT emp_id, first_name, salary FROM employees ORDER BY salary;
```

Result:

emp_id	first_name	salary
101	John	5000
103	Bob	6000
102	Alice	7000

Note: Here, by default 'salary' is sorted in Ascending order.

✅ Example 2: Sort by Salary (Descending)

```
SQL>SELECT emp_id, first_name, salary FROM employees ORDER BY salary DESC;
```

Result:

emp_id	first_name	salary
102	Alice	7000
103	Bob	6000
101	John	5000

✅ Example 3: Sort by Multiple Columns

```
SQL>SELECT emp_id, first_name, salary FROM employees ORDER BY salary DESC, first_name ASC;
```

- First sorts by salary in descending order.
- Then sorts by first name in ascending order **only if salaries are equal**.

Result:

EMPID	FIRSTNAME	SALARY
102	Alice	7000
103	Bob	6000
101	John	5000

✅ Example 4: Using Column Numbers

You can also sort by **column positions** as follow-

```
SQL>SELECT emp_id, first_name, salary FROM employees ORDER BY 3 ASC;
```

-- Here, data is sorted in Ascending order of 3rd column (salary)

Result:

EMPID	FIRSTNAME	SALARY
101	John	5000
103	Bob	6000
102	Alice	7000

2) GROUP BY Clause:

- The GROUP BY clause in Oracle SQL is used to group rows i.e records that have the same values in specified columns into summary rows, like finding the **total, average, count**, etc., for each group.
- GROUP BY clause typically used with **aggregate functions** like SUM(),AVG(), COUNT(),MAX() MIN() etc.

- **Syntax:**

SQL>SELECT column1, aggregate_function(column2) FROM table_name GROUP BY column1;

- column1 is the column you want to group by.
- aggregate_function(column2) is the calculation you want to perform on another column.

Example: Using GROUP BY

Suppose we have a table **SALES** with the following structure and data:

```
SQL> SELECT * FROM SALES;
```

SALE_ID	SALESMAN	REGIO	AMOUNT
1	KIRAN	EAST	500
2	AMIT	WEST	300
3	KIRAN	EAST	700
4	AMIT	WEST	200
5	VIRAJ	EAST	600

✓ Query: Total Sales amount by Salesman

```
SQL> SELECT salesman, SUM(AMOUNT) AS TOTAL_SALES FROM SALES GROUP BY SALESMAN;
```

SALESMAN	TOTAL_SALES
KIRAN	1200
VIRAJ	600
AMIT	500

✓ Query: Total Sales by Region

```
SQL> SELECT REGION, SUM(AMOUNT) AS TOTAL_SALES FROM SALES GROUP BY REGION;
```

REGIO	TOTAL_SALES
WEST	500
EAST	1800

✓ Query: Average sale amount by Region

```
SQL> SELECT REGION,AVG(AMOUNT) FROM SALES GROUP BY REGION;
```

REGIO	AVG(AMOUNT)
WEST	250
EAST	600

✓ Query: Minimum sale amount by Salesman.

```
SQL> select salesman,min(amount) from sales group by salesman;
```

SALESMAN	MIN(AMOUNT)
KIRAN	500
VIRAJ	600
AMIT	200

❖ You can use GROUP BY with **multiple columns** like-

I) Total sale amount by salesman and region.

```
SQL> SELECT REGION, SALESMAN, SUM(AMOUNT) FROM SALES GROUP BY REGION, SALESMAN;
```

REGIO	SALESMAN	SUM(AMOUNT)
EAST	KIRAN	1200
WEST	AMIT	500
EAST	VIRAJ	600

II) Region counts by salesman and region.

```
SQL> select salesman,region,count(region) from sales group by salesman,region;
```

SALESMAN	REGIO	COUNT(REGION)
VIRAJ	EAST	1
AMIT	WEST	2
KIRAN	EAST	2

3) HAVING Clause:

- In **Oracle SQL**, the **HAVING clause** is used to filter groups of rows i.e. records created by the **GROUP BY** clause.
- HAVING clause works like a WHERE clause **but it works on aggregated or grouped data.**
- **Note-**
 - WHERE clause filters individual rows before grouping where as HAVING clause filters groups (aggregated results) **after grouping.**
 - We can use HAVING clause with aggregate functions like COUNT(), SUM(), AVG(), MIN(), MAX().

- **Syntax:**

```
SQL>SELECT column1, aggregate_function(column2) FROM table_name GROUP BY column1  
HAVING condition;
```

Here,

- **column1** is the column you want to group by.
- **aggregate_function(column2)** is the calculation you want to perform on another column.
- **condition** is condition used to filter the group.

Example-Suppose we have a table **emp_dept**:

```
SQL> SELECT * FROM emp_dept;
```

EMP_ID	DEPT	SALARY
1	HR	30000
2	HR	40000
3	IT	60000
4	IT	55000
5	IT	45000
6	SALES	25000

1. Find total salary per department (use of GROUP BY clause)

```
SQL> SELECT DEPT, SUM(SALARY) AS TOT_SALARY FROM emp_dept GROUP BY DEPT;
```

DEPT	TOT_SALARY
IT	160000
HR	70000
SALES	25000

2. Now, we use HAVING to filter groups by condition-

👉 Example: Show only departments where **total salary > 50000**

```
SQL> SELECT DEPT, SUM(SALARY) AS TOTAL_SALARY FROM emp_dept GROUP BY DEPT HAVING SUM(SALARY) > 50000;
```

DEPT	TOTAL_SALARY
IT	160000
HR	70000

Here, (SALES is excluded because its total = 25000, which is not > 50000)

3. We can also Combine WHERE + HAVING as follow-

👉 Example: Show only departments with employees earning salary > 30000, and total salary > 100000

```
SQL> SELECT DEPT, SUM(SALARY) AS TOT_SALARY FROM emp_dept WHERE SALARY > 30000  
2 GROUP BY DEPT HAVING SUM(SALARY) > 100000;
```

DEPT	TOT_SALARY
IT	160000

4) ON DELETE CASCADE clause:

- The **ON DELETE CASCADE** clause in **Oracle SQL** is used in **foreign key constraints**.
- This clause tells Oracle:
👉 *"If a row in the parent table is deleted, automatically delete all matching rows in the child table."*
- **ON DELETE CASCADE** clause ensures **referential integrity** and avoids **orphan records** in child tables.

🔑 **Note-**

1. **ON DELETE CASCADE** clause used with **FOREIGN KEY** definition.
2. **ON DELETE CASCADE** clause allows automatically deletion of rows in **child table** when the **parent row** is deleted.
3. **ON DELETE CASCADE** clause useful in **master-detail (parent-child)** relationships.

Syntax- Creating Table with ON DELETE CASECADE clause

```
SQL>CREATE TABLE child_table (  
  Col1 datatype,  
  Col2 datatype,  
  ...,  
  CONSTRAINT fk_name FOREIGN KEY (child_column)  
  REFERENCES parent_table(parent_column)  
  ON DELETE CASCADE  
);
```

Example-

1) Create PARENT table with PRIMARY KEY constraint-

```
SQL> create table parent (pid number PRIMARY KEY, pname varchar2(10));  
Table created.
```

2) Create CHILD table with FOREIGN KEY constraint along with ON DELETE CASCADE clause

```
SQL> create table child (cid number ,cname varchar2(10),pid REFERENCES parent(pid) ON DELETE CASCADE);  
Table created.
```

3) Inserting records into PARENT table-

```
SQL> insert into parent values(&pid, '&pname');  
Enter value for pid: 1  
Enter value for pname: KISHOR  
old 1: insert into parent values(&pid, '&pname')  
new 1: insert into parent values(1, 'KISHOR')  
  
1 row created.  
  
SQL> /  
Enter value for pid: 2  
Enter value for pname: VIJAY  
old 1: insert into parent values(&pid, '&pname')  
new 1: insert into parent values(2, 'VIJAY')  
  
1 row created.
```

4) Inserting records into CHILD table-

```
SQL> insert into child values(&cid, '&cname', &pid);  
Enter value for cid: 101  
Enter value for cname: SAHIL  
Enter value for pid: 1  
old 1: insert into child values(&cid, '&cname', &pid)  
new 1: insert into child values(101, 'SAHIL', 1)  
  
1 row created.  
  
SQL> /  
Enter value for cid: 102  
Enter value for cname: KOMAL  
Enter value for pid: 2  
old 1: insert into child values(&cid, '&cname', &pid)  
new 1: insert into child values(102, 'KOMAL', 2)  
  
1 row created.
```


5) Display records of PARENT and CHILD tables-

```
SQL> select * from parent;
```

PID	PNAME
1	KISHOR
2	VIJAY

```
SQL> select * from child;
```

CID	CNAME	PID
101	SAHIL	1
102	KOMAL	2

6) DELETE record from PARENT table-

```
SQL> delete from parent where pid=1;
```

```
1 row deleted.
```

Note- Here, parent record is deleted that refer to child record also this is because of **ON DELETE CASCADE** clause.

7) Display records of PARENT and CHILD tables-

```
SQL> select * from parent;
```

PID	PNAME
2	VIJAY

```
SQL> select * from child;
```

CID	CNAME	PID
102	KOMAL	2

8. Effect

- The row with pid = 1 is deleted from PARENT table.
- Automatically, all child's with pid = 1 are also deleted from CHILD table.

So now, only KOMAL (pid = 2) remains in the CHILD table.



Without ON DELETE CASCADE

If you try to delete a parent that still has child's, Oracle will throw an integrity constraint error (child records exist).

With ON DELETE CASCADE, Oracle takes care of child rows automatically.

Theory Assignment-II

Short Answer Questions

- 1 What is the difference between DDL and DML commands?
- 2 Define transaction in SQL. Give two examples of TCL commands.
- 3 What is the purpose of the ON DELETE CASCADE clause in Oracle SQL?
- 4 Write syntax to rename a table using the ALTER command.
- 5 What is the difference between TRUNCATE and DELETE commands?
- 6 Define Foreign Key with an example.
- 7 What are aggregate functions in SQL? Give two examples.
- 8 Explain BETWEEN and LIKE operators with examples.
- 9 What is the role of the COMMIT statement?
- 10 Write syntax for creating a table with a PRIMARY KEY constraint.

Long Answer Questions

- 1 Explain different categories of SQL commands: DDL, DML, DCL, TCL with suitable examples.
- 2 Discuss various Integrity Constraints in SQL: Primary Key, Foreign Key, Unique, Check, Not Null with syntax and examples.
- 3 Explain operators in SQL with examples: Arithmetic, Comparison, Logical, Set, and Special operators.
- 4 What are built-in functions in SQL? Explain numeric, string, and date functions with examples.
- 5 Describe Clauses in SQL: WHERE, ORDER BY, GROUP BY, HAVING, DISTINCT with examples.
- 6 Explain transaction control in SQL with COMMIT, ROLLBACK, SAVEPOINT, and SET TRANSACTION commands. Give examples.
- 7 Write detailed notes on ALTER TABLE command with examples of adding, modifying, renaming columns and adding/dropping constraints.
- 8 Differentiate between DELETE, TRUNCATE, and DROP commands with examples.

Multiple Choice Questions (MCQs)

- 1 Which of the following is a DDL command? a) SELECT b) INSERT c) CREATE d) UPDATE
Answer: c) CREATE
- 2 Which SQL command is used to remove all rows but keep the table structure? a) DROP b) DELETE c) TRUNCATE d) REMOVE **Answer: c) TRUNCATE**
- 3 Which of the following is a valid TCL command? a) GRANT b) SAVEPOINT c) REVOKE d) ALTER **Answer: b) SAVEPOINT**
- 4 A Foreign Key is used to: a) Uniquely identify each record in a table b) Link two tables together
c) Enforce NOT NULL condition d) Automatically increment column values **Answer: b) Link two tables together**
- 5 Which of the following functions returns the length of a string? a) SUBSTR() b) LENGTH() c) INSTR() d) TRIM() **Answer: b) LENGTH()**
- 6 The HAVING clause is used with: a) WHERE b) GROUP BY c) ORDER BY d) DISTINCT
Answer: b) GROUP BY
- 7 The ON DELETE CASCADE option ensures: a) Parent rows cannot be deleted if child rows exist b) Deleting a parent row automatically deletes matching child rows c) Child table is locked when parent is deleted d) None of the above **Answer: b) Deleting a parent row automatically deletes matching child rows**